

第九章 分枝限界法



学习要点

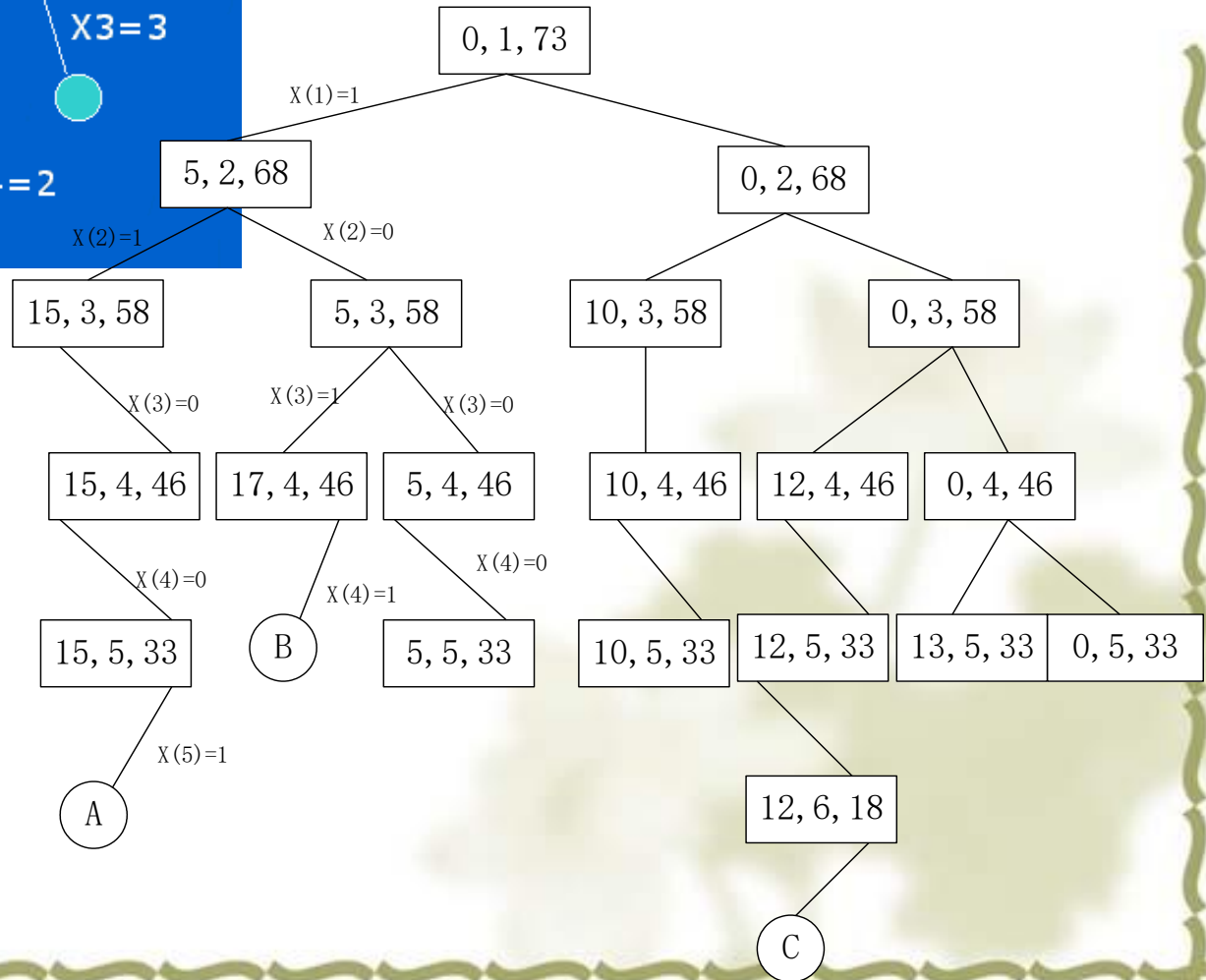
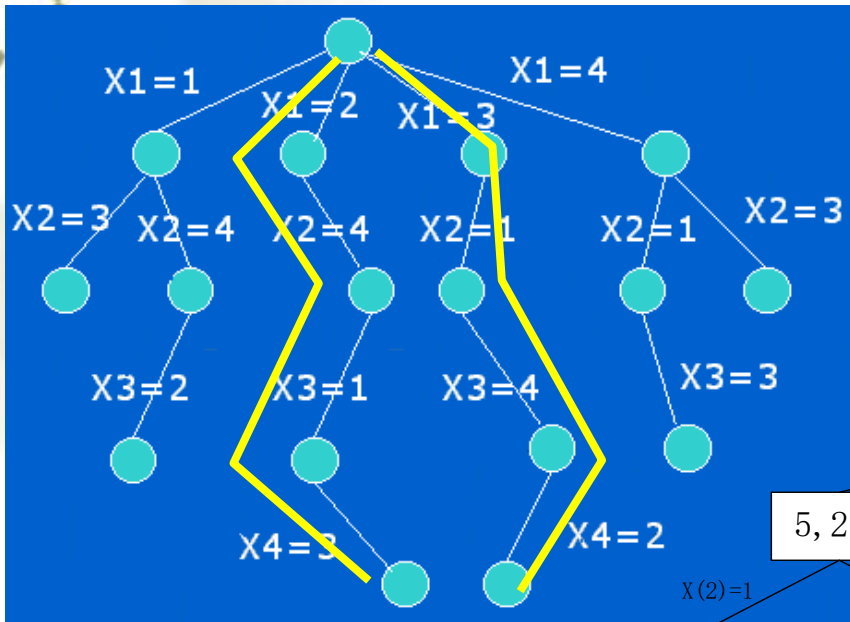
- 理解分枝限界法的概念。着重讨论可以用分枝限界法求解的问题的一般特征。
- 掌握分枝限界算法的基本要素
- 理解分枝限界算法的一般理论
- 通过应用范例学习。

一般方法；

- FIFO、LIFO、LC检索；
- 分枝限界；

对于状态树的搜索，回溯法是深度优先的搜索。回顾上一章，就可看出对于某一类问题，回溯法的搜索效率较低。

例如求最优解问题，不需要搜索整个空间树。有时采用其他搜索策略可能会更有效，如用最小代价去搜索等。



9.1 一般方法

搜索方式:

深度优先搜索

广度优先搜索

最小代价搜索

分支限界法

存储结构:

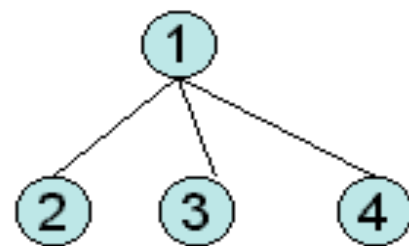
——栈结构

——队列结构

——队列结构

FIFO检索

LIFO检索



$L=(2,3,4)$

这两种方法都是对当前**扩展结点E**的所有儿子进行检测，满足约束的儿子结点放入**活结点表**中。该扩展结点E完成使命，成为**死结点**。再取活结点.....

FIFO: 取2作下一个扩展结点

LIFO: 取4作下一个扩展结点

9.1.1 分枝限界法

➤采用广度优先产生状态空间树的结点，并使用剪枝函数的方法称为**分枝限界法**。

➤按照广度优先原则，在生成当前扩展结点的全部儿子后，并将它们一一加入活结点表中，再从活结点表另选一活结点扩展其结点的儿子。

➤不同的活结点表形成不同的分枝限界法。

FIFO分枝限界法—先进先出

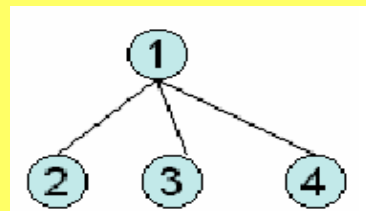
LIFO分枝限界法—后进先出

LC 分枝限界法—最小代价

1)扩展结点

2)活结点

3)死结点



三种不同的活结点表，规定了从活结点表中选取下一个E-结点的次序。

LC分枝限界法将选取具有最高优先级的活结点出队列，成为新的E-结点。

9.1.1 分枝限界法

➤采用广度优先产生状态空间树的结点，并使用剪枝函数的方法称为分枝限界法。

➤按照广度优先产生结点，并展将它们一一其结点

回溯法的求解目标：在状态空间树上找出满足约束条件的所有解。

➤不同

分枝限界法的求解目标：找出满足约束条件的一个解，或是在满足约束条件的解中找出最优解。

FI
LI
LC

三种不同
一个E-结点时以
LC分枝限界法将选取具有最高优先级的活结点出队列，成为新的E-结点。

例：4-皇后问题的状态空间树的FIFO搜索。最初E-为1，产生儿子2,3,4,5,放入活结点表。

L=(2,3,4,5)

L=(3,4,5,6,7)

L=(4,5,6,7,8)

L=(5,6,7,8,9)

L=(6,7,8,9,10,11)

L=(8,9,10,11,12)

L=(9,10,11,12,13)

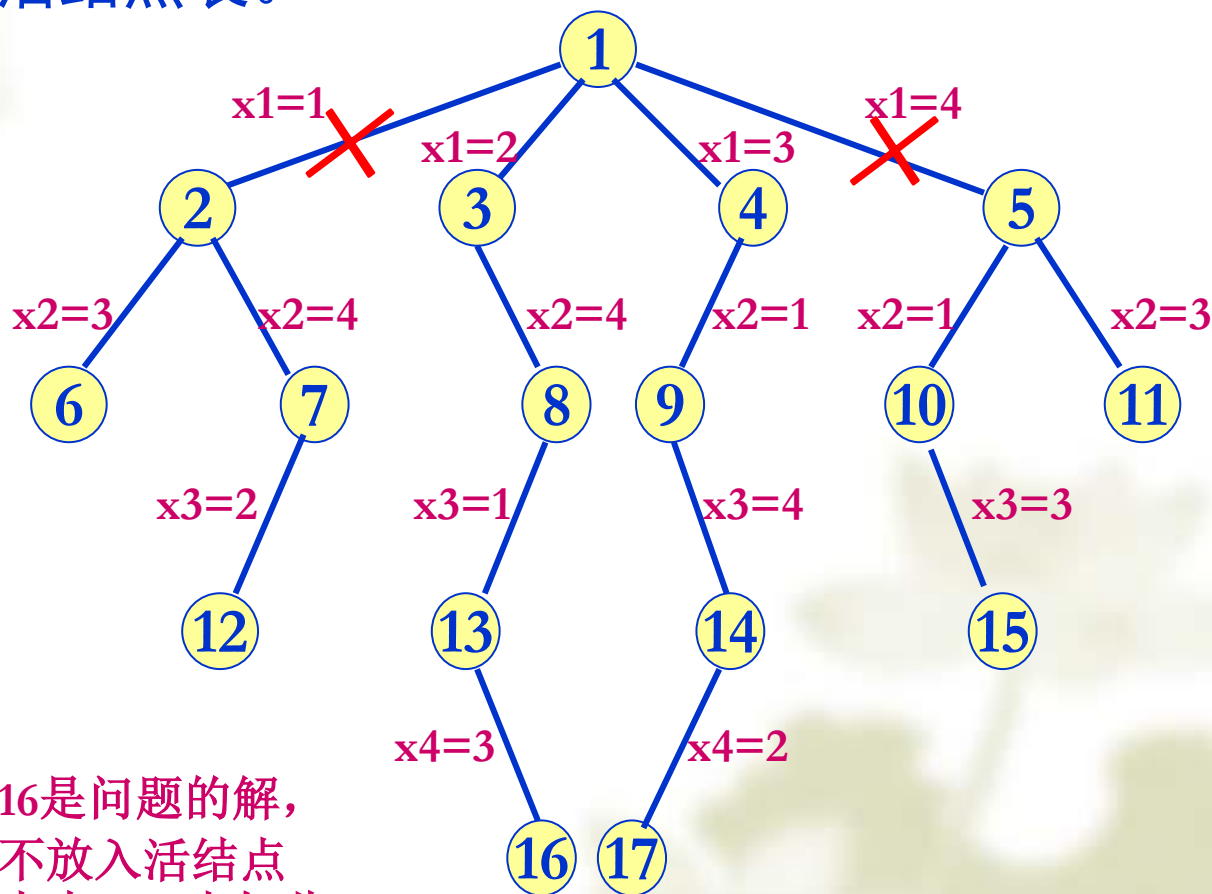
L=(10,11,12,13,14)

L=(11,12,13,14,15)

L=(14,15)

L=(15)

L=()



这样的搜索并未提高效率，产生的结点不比回溯法少。
希望能“智能”的选择搜索路线，而把不需要的剪掉。

算法框架：

procedure branch bound(T) //T为空间树根

E=T

loop

begin

for(对结点E的每个 **不受限的** 儿子)

构造E的孩子结点x

if(x是一个答案结点)

输出从x到t的一条路经; **return**;

x进入活结点表

repeat

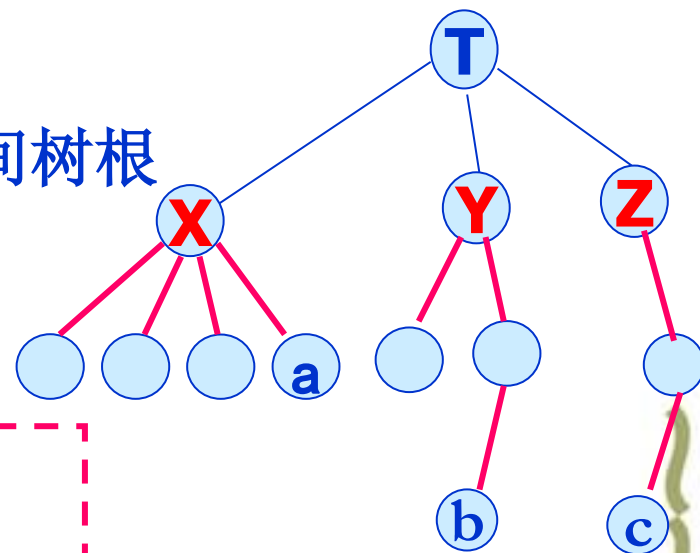
if (活结点表空否) //不再有活结点
无答案结点; **return**;//搜索失败

输出(取出)一个E-结点为E

end

repeat

end



算法在求得可行解后停止，

增加一些处理，如全局变量、相应计算等，可求得问题的最优解。

FIFO、LIFO分枝限界法从活结点表中选择下一个活结点，作为新E-结点的做法是**盲目的**，它们只是机械地按照FIFO或LIFO原则选取下一个活结点。

这种选择规则对于有可能快速检索到一个答案结点的结点没有给出任何优先权。

回溯法和分枝限界法都可以使用约束函数来剪去不含答案结点的分枝，并都可使用限界函数剪去那些不含最优解的分枝。

对活结点使用一个“有智力的”函数来选取下一个E-结点往往可以加快到达一个答案结点的检索速度。

FIFO、LIFO分点，作为新E-结点的FIFO或LIFO原则选这种选择规则对点没有给出任何优先权。

对于较少的 n ，可以人为的智能的去做，大量 n 时需找出一些规则，制定一些标准，尽管该标准有局限。可以采用最小代价的搜索方法。它可以去掉一些搜索的盲目性。

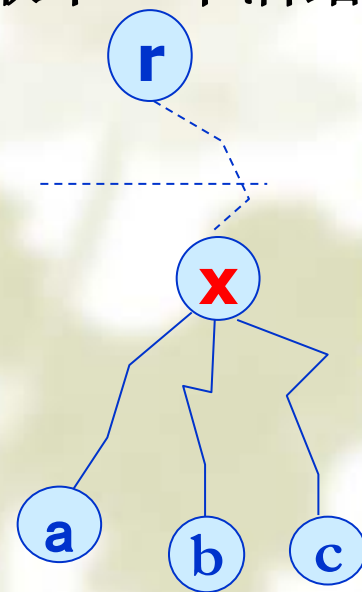
回溯法和分枝限界法都可以使用约束函数来剪去不含答案结点的分枝，并都可使用限界函数剪去那些不含最优解的分枝。

对活结点使用一个“有智力的”函数来选取下一个E-结点往往可以加快到达一个答案结点的检索速度。

9.1.2 最小代价搜索(LC搜索)

可根据每个活结点的优先权进行选择。

- 使用LC分枝限界法来加速搜索到答案结点。
- 如果将一个问题状态的优先权定义为“在状态空间树上搜索一个答案结点所需的代价”，使得搜索代价小的活结点优先被检测，理论上应能较快地搜索到一个答案结点。
- 对活结点施以“有智力的”排序(评价)函数来选取下一个活结点。
- 一个答案结点 x 的搜索代价 $\text{cost}(x)$ 定义为：
从根结点开始，直到搜索到 x 为止所耗费的搜索时间。



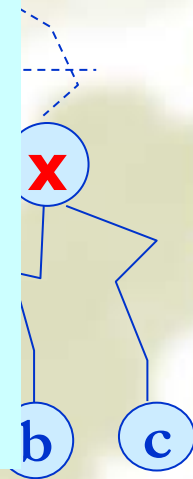
9.1.2 最小代价搜索(LC搜索)

可根据每个活结点的优先权进行选择。

- 使用LC分枝限界法来加速搜索到答案结点。
- 如果将一个问题状态的优先权定义为“在状态空间树上搜索一个答案结点所需的代价”，使得**搜索代价小的活结点优先被检测**，理论上应能较快地搜索到一个答案结点。
- 对活结点施以“有智力的”排序(评价)函数来选取下一个活结点。

对于任一结点 x ，要付出的代价可以使用以下标准来度量：

- (1) 在生成一个答案结点之前，子树 x 需要生成的结点数；
- (2) 在子树 x 中离 x 最近的那个答案结点到 x 的路径长度。



四个相关函数：

1. 代价函数 $c(\bullet)$ ，又称为结点的成本函数。其定义：

如果 X 是答案结点，则 $C(X)$ 是从根结点到 X 的搜索成本；

如果 X 不是答案结点且子树 X 不包含任何答案结点，则 $C(X) = \infty$ ；

如果 X 不是答案结点但子树 X 包含任何答案结点，则 $C(X)$ 等于子树 X 上具有最小搜索成本的答案结点的成本。

设 a, b, c 是答案结点，此时对应的代价： $a < b < c$

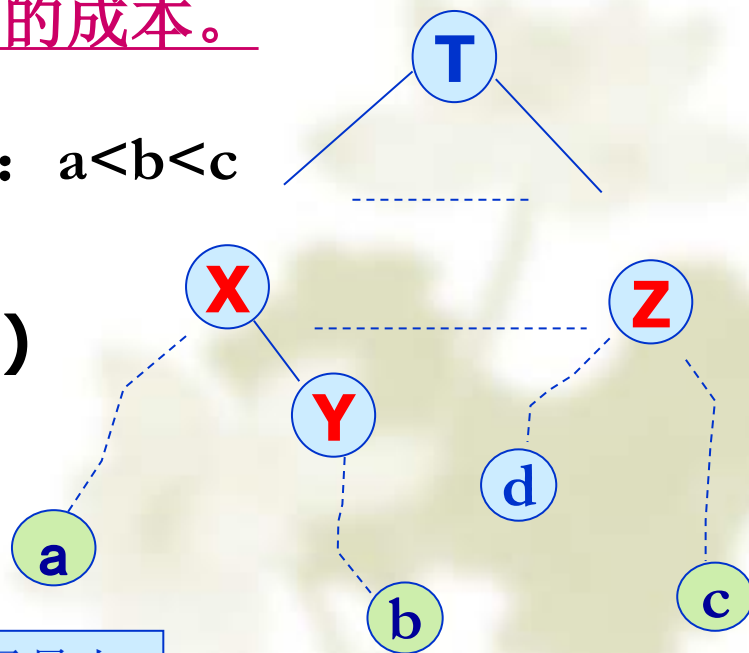
则

$$C(T) = \dots = C(X) = \dots = C(a) = \text{cost}(a)$$

$$C(Y) = \dots = C(b) = \text{cost}(b)$$

$$C(Z) = \dots = C(c) = \text{cost}(c)$$

$$\dots = C(d) = \infty$$



这里的=是选择了最小

四个相关函数：

I. 代价函数 $c(\bullet)$ ，又称为结点

如果 X 是答案结点，则 $C(X)$ 是从根结

如果 X 不是答案结点且子树 X 不包含

如果 X 不是答案结点但子树 X 包含任何答案结点，则 $C(X)$ 等于子树 X 上具有最小搜索成本的答案结点的成本。

但要指出的是：要得到结点成本函数 $C(\cdot)$ 所用的计算工作量与解原问题具有相同的复杂度，所以要得到精确的成本函数一般是不现实的。

设 a, b, c 是答案结点，此时对应的代价： $a < b < c$

则

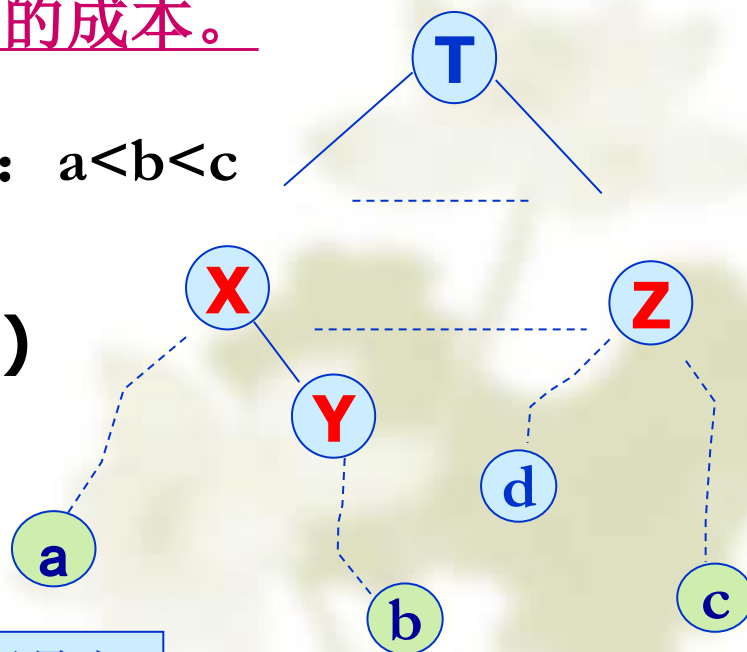
$$C(T) = \dots = C(X) = \dots = C(a) = \text{cost}(a)$$

$$C(Y) = \dots = C(b) = \text{cost}(b)$$

$$C(Z) = \dots = C(c) = \text{cost}(c)$$

$$\dots = C(d) = \infty$$

这里的 $=$ 是选择了最小



II. 相对代价函数 $g(\bullet)$ ，度量标准：

- (1) 在生成一个答案结点之前，子树X上需要生成的结点数目；
- (2) 在子树X上离X最近的那个答案结点到X的路径长度。

采用(1)，总是生成最小数目的结点；

采用(2)，则要成为E-结点的结点只是由根到最近的那个答案结点路径上的那些结点。

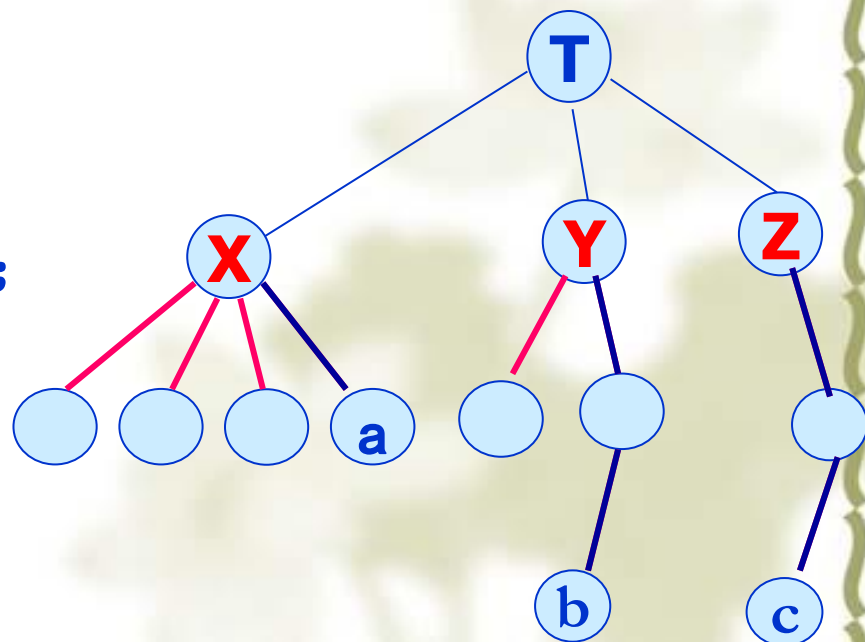
设a,b,c是答案结点，

采用(1)， $g(X)=4, g(Y)=3, g(Z)=2$,

此时算法首先找到答案结点c；

采用(2)， $g(X)=1, g(Y)=g(Z)=2$,

算法首先找到答案结点a。



II. 相对代价函数 $g(\bullet)$, 度量标准

- (1) 在生成一个答案结点之前,
- (2) 在子树X上离X最近的那个答案

采用(1), 总是生成最小数目的结点

采用(2), 则要成为E-结点的结点

近的那个答案结点路径上的

可以看出: 要得到 $g(\cdot)$ 函数, 所用的计算工作量与 $c(\cdot)$ 及解原问题具有相同的复杂度, 所以要得到相对成本函数也是比较困难的。

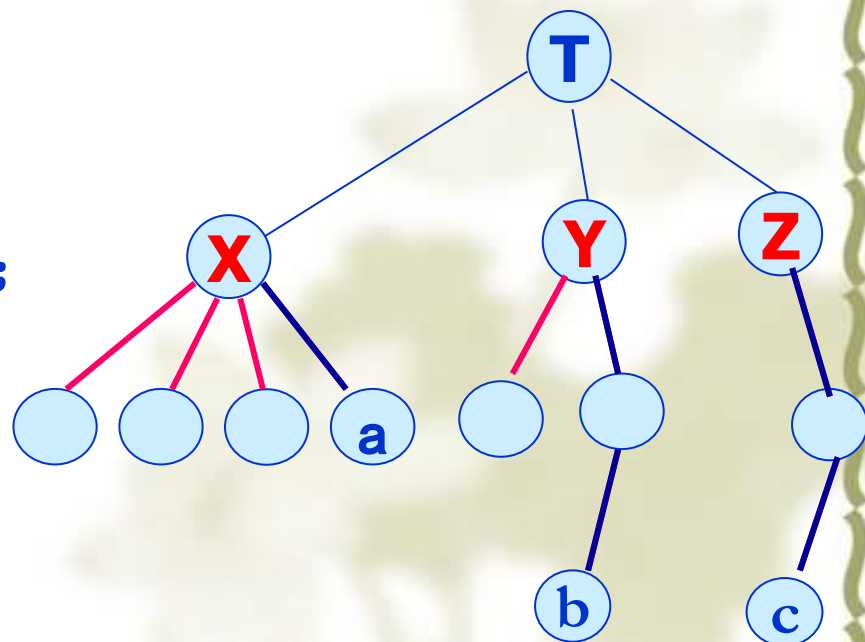
设 a, b, c 是答案结点,

采用(1), $g(X)=4, g(Y)=3, g(Z)=2$,

此时算法首先找到答案结点 c ;

采用(2), $g(X)=1, g(Y)=g(Z)=2$,

算法首先找到答案结点 a 。



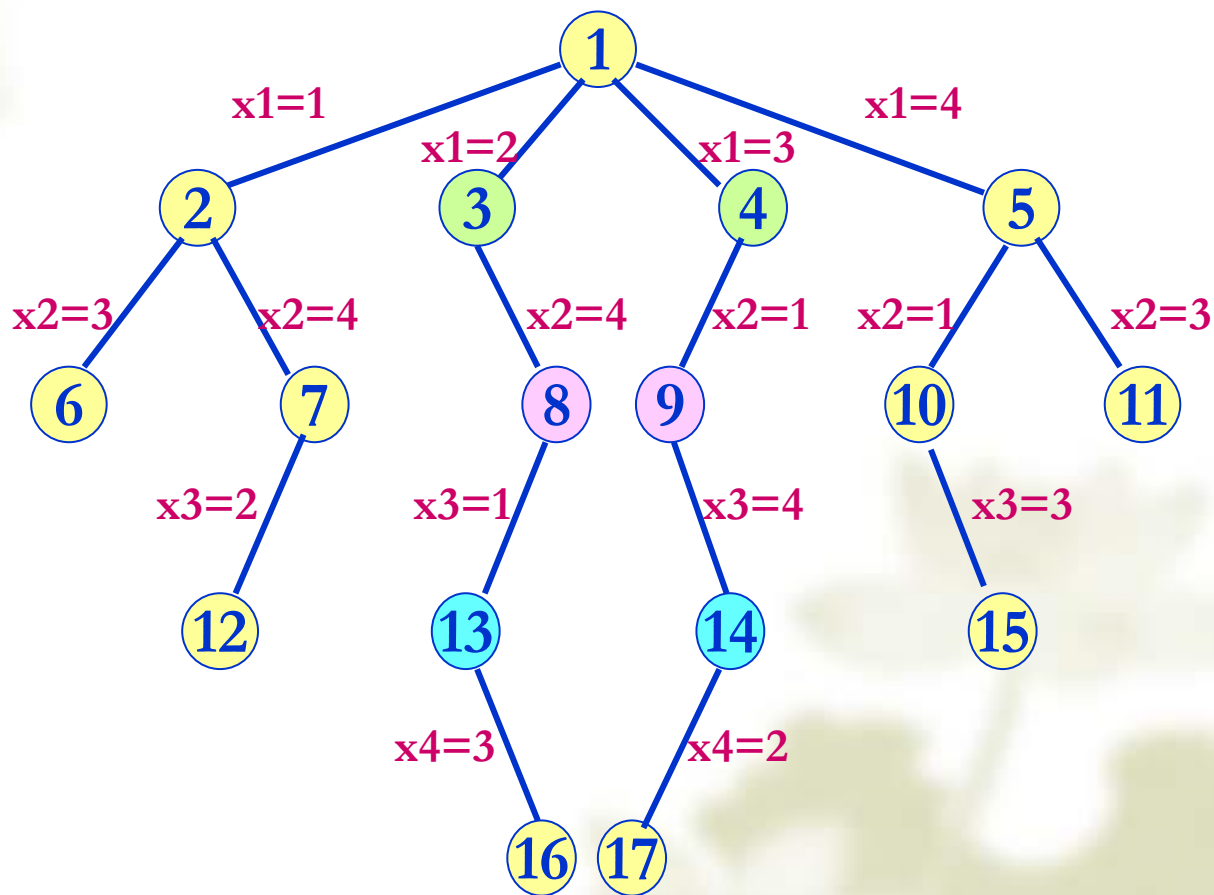
1级

2级

3级

4级

5级



?

树的代价为 **4**：由根到答案付出的代价

代价为 3：由3、4到答案付出的代价

1级

2级

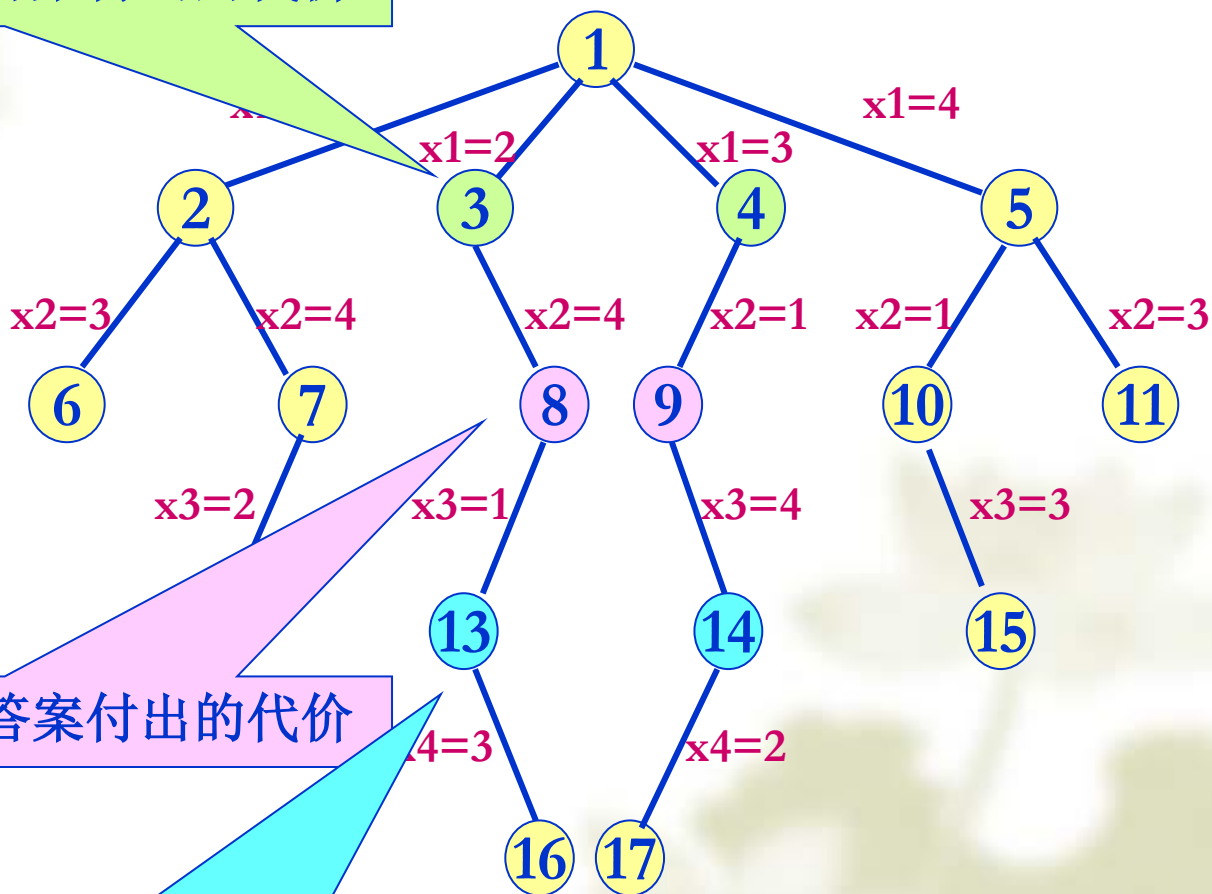
3级

4级

代价为 2：由8、9到答案付出的代价

5级

代价为 1：由13、14到答案付出的代价





树的代价为 **4**：由根到答案付出的代价

代价为 3：由3、4到答案付出的代价

1级

以这些代价作为选择下一个E-结点的依据，则E-结点依次为1, 3, 8, 13, 16。

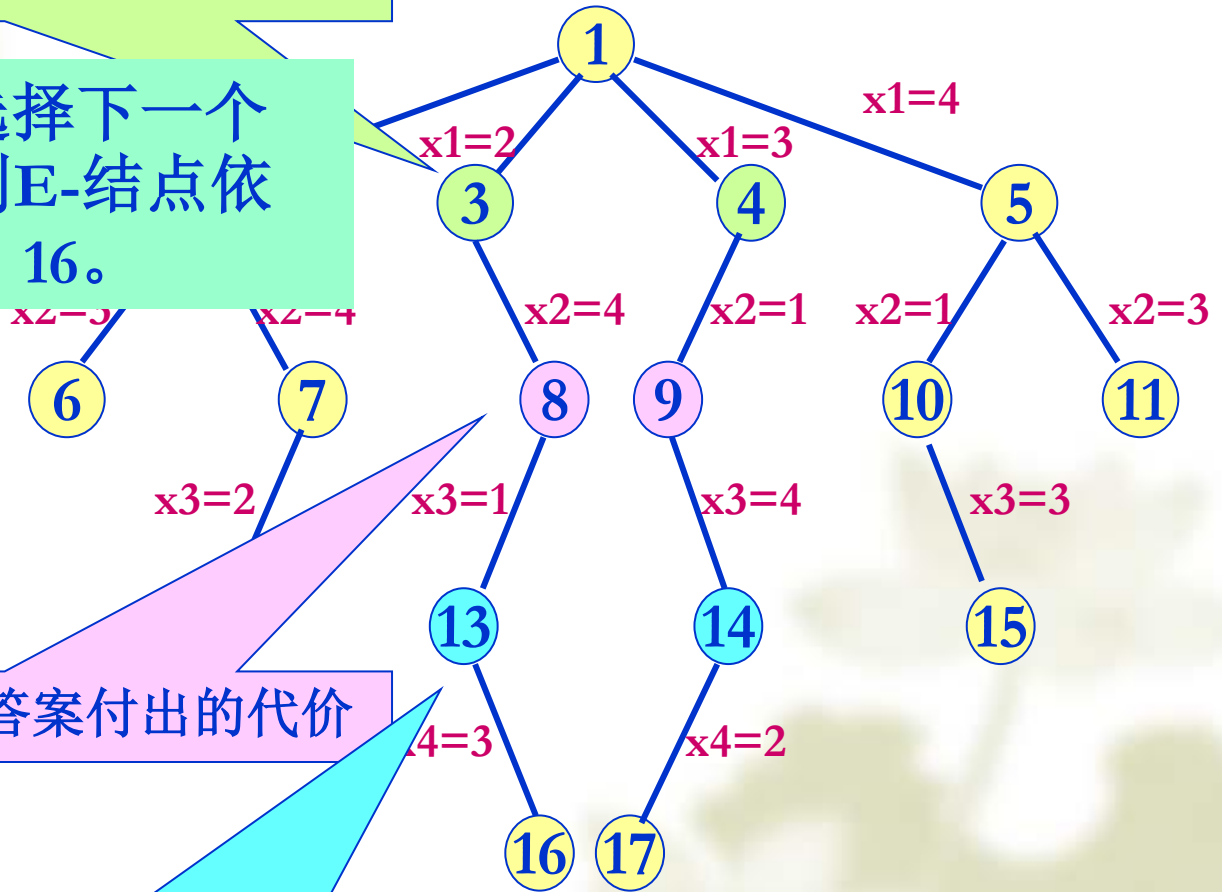
3级

4级

代价为 2：由8、9到答案付出的代价

5级

代价为 1：由13、14到答案付出的代价





树的代价为 **4**：由根到答案付出的代价

代价为 3：由3、4到答案付出的代价

1级

以这些代价作为选择下一个E-结点的依据，则E-结点依次为1, 3, 8, 13, 16。

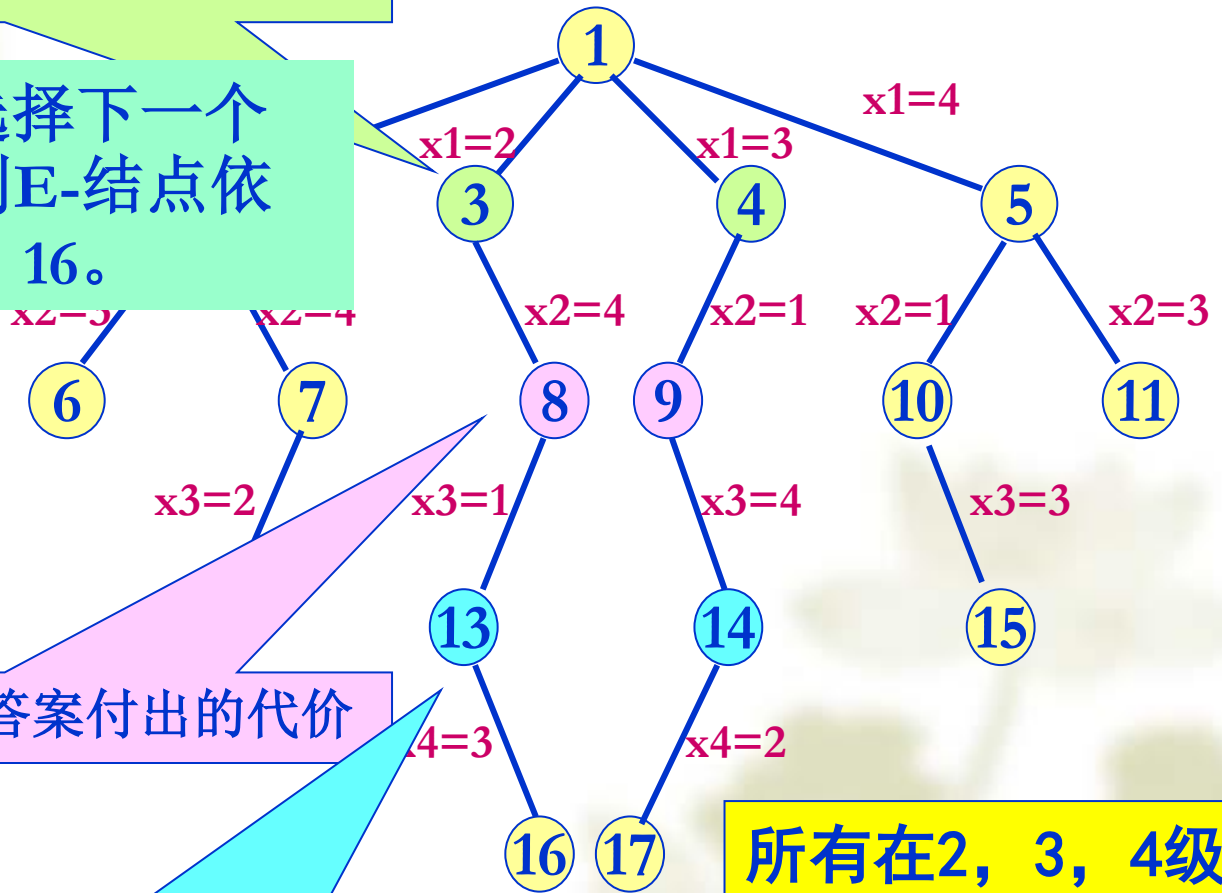
3级

4级

代价为 2：由8、9到答案付出的代价

5级

代价为 1：由13、14到答案付出的代价



所有在2, 3, 4级上的剩余结点的代价应分别大于3, 2和1。

树的代价为 **4**：由根到答案付出的代价

代价为 3：由3、4到答案付出的代价

1级

以这些代价作为选择下一个E-结点的依据，则E-结点依次为1, 3, 8, 13, 16。

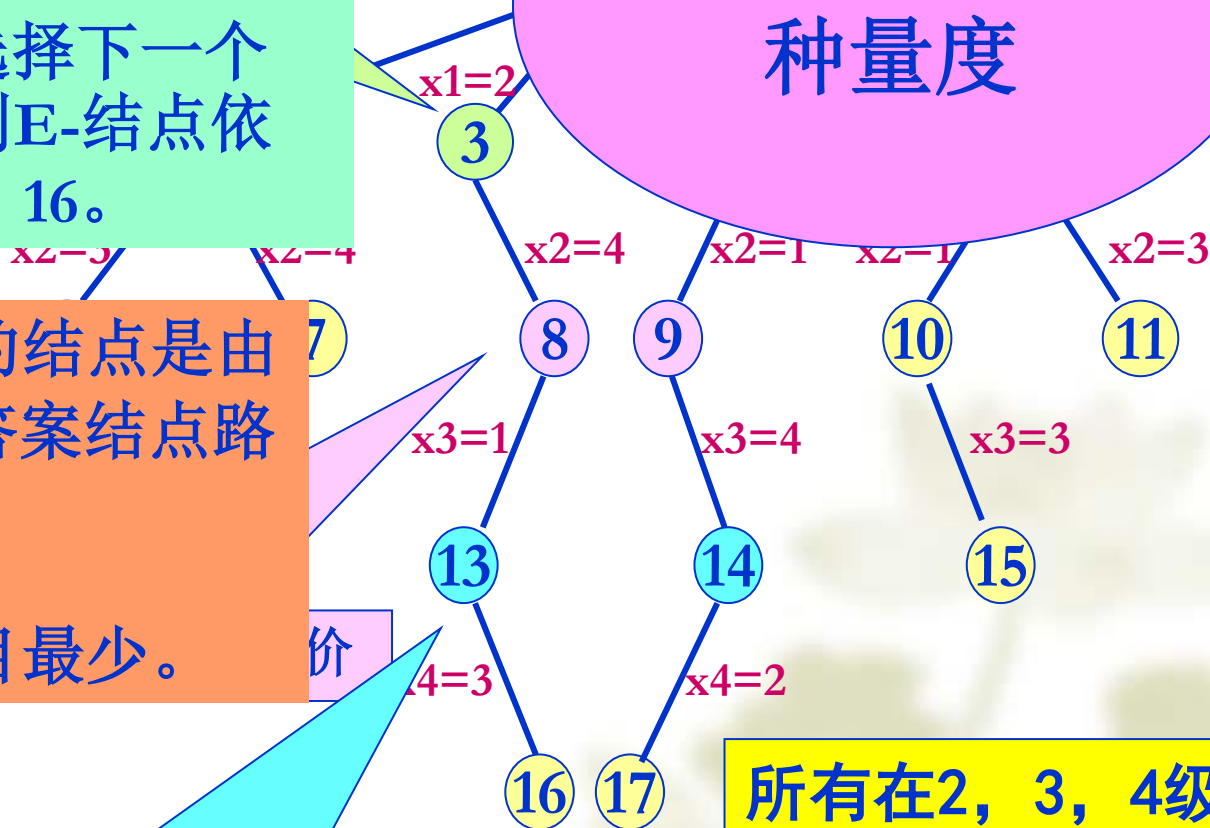
所以成为E-结点的结点是由根到最近的那个答案结点路径上的那些结点。

产生的E-结点数目最少。

5级

代价为 1：由13、14到答案付出的代价

用了第二
种量度



所有在2, 3, 4级上的剩余结点的代价应分别大于3, 2和1。

III. 相对代价估值函数 $\hat{g}$

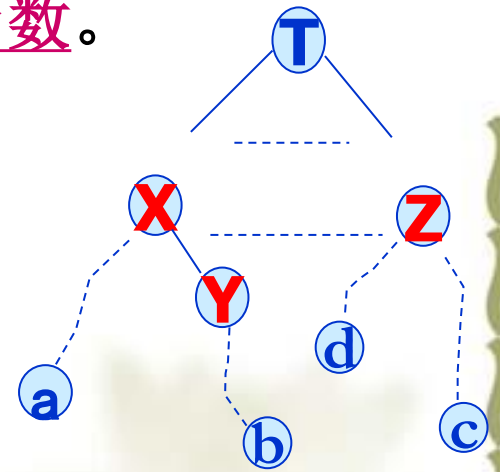
由于构建 $c(\cdot)$ 和 $g(\cdot)$ 的困难，常采用它们的估计值来构建评价函数。

$\hat{g}(X)$ 作为 $g(X)$ 的估值，用于估计结点 X 的相对代价，它是由 X 到达一个答案结点所需代价的估计函数。

设 $\hat{g}(X)$ 有如下特性：

如果 Y 是 X 的孩子，则有 $\hat{g}(Y) \leq \hat{g}(X)$ 。
—— Y 可能更接近答案结点

✱但是单纯使用函数 $\hat{g}(\cdot)$ 并不合适，它会导致算法偏向于作纵深检查。



例：如果 $g(Y) > g(X)$ ，但因为 $\hat{g}$ 是估计函数，故也许有 $\hat{g}(Y) < \hat{g}(X)$ ，此时会导致在 Y 为根的子树上向纵深搜索；又如 $\hat{g}(Z) < \hat{g}(Y)$ 时，而搜索结点 c ，但偏离了搜索代价最小的结点 a 。

IV. 代价估值函数 $\hat{c}(\bullet)$

$\hat{c}(X)$ 是代价估计函数，它由两部分组成：从根到X的代价 $f(h(X))$ 和从X到答案结点的估计代价 $\hat{g}(X)$ ，即

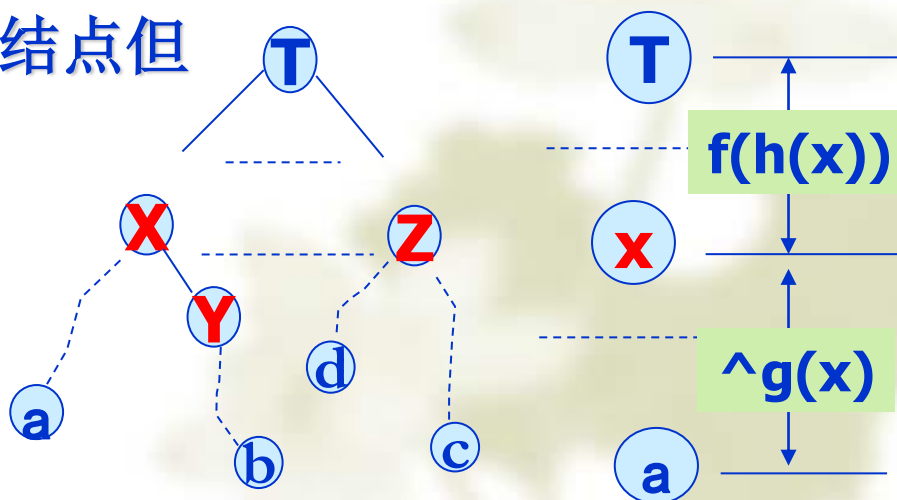
$$\hat{c}(X) = f(h(X)) + \hat{g}(X)$$

用 $\hat{c}(X)$ 来减少算法作偏向于纵深检查的可能性，它可以使算法在结点X、Y和Z之间优先检索更靠近答案结点但又离根较近的结点X。

一般可令 $f(h(X))$ 等于X在树中的层次($f(\cdot)$ 不都等于0)。
 $f(h(X))$ 是一个非降函数。

解决方法：不仅考虑结点X到一个答案结点的估计成本值，还应考虑由根结点到结点X的成本。引入 $\hat{c}(\cdot)$ 。

有 $\hat{c}(x) \leq c(x)$



最小成本检索的搜索策略：

以优先权队列作为活结点表，以代价估计函数 $\hat{c}(\bullet)$ 作为选择下一个扩展结点的评价函数，即总是选取 $\hat{c}(\bullet)$ 值最小的活结点为下一个E-结点。——简称LC-检索。

LC分枝限界法采取LC检索方法。

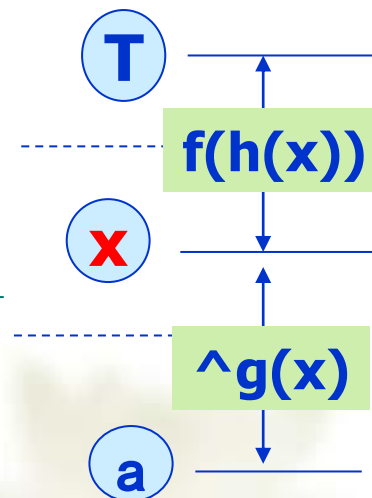
——这里也使用了剪枝函数

再来考察代价估计函数：

$$\hat{c}(X) = f(h(X)) + \hat{g}(X)$$

- 如果 $\hat{g}(\bullet) \equiv 0$ ，且 $f(h(X))$ 等于X在树中的层次，则LC-检索表现出宽度优先搜索特性，成为FIFO检索。
- 如果 $f(\bullet) \equiv 0$ ，则 $\hat{c}(X) = \hat{g}(X)$ ，LC-检索表现出深度优先搜索特性，成为D-检索。

——这种做法不恰当： $\hat{g}(X)$ 是 $g(X)$ 的一个估计值，过分向纵深搜索有时并不能更快接近答案结点，甚至会偏离答案结点。



最小成本检索的搜索

以优先权队列作为活
择下一个扩展结点的评
结点为下一个E-结点。

LC分枝限界法采取

——这里也使用了

再来考察代价估计函数。

$$\hat{c}(X) = f(h(X)) + \hat{g}(X)$$

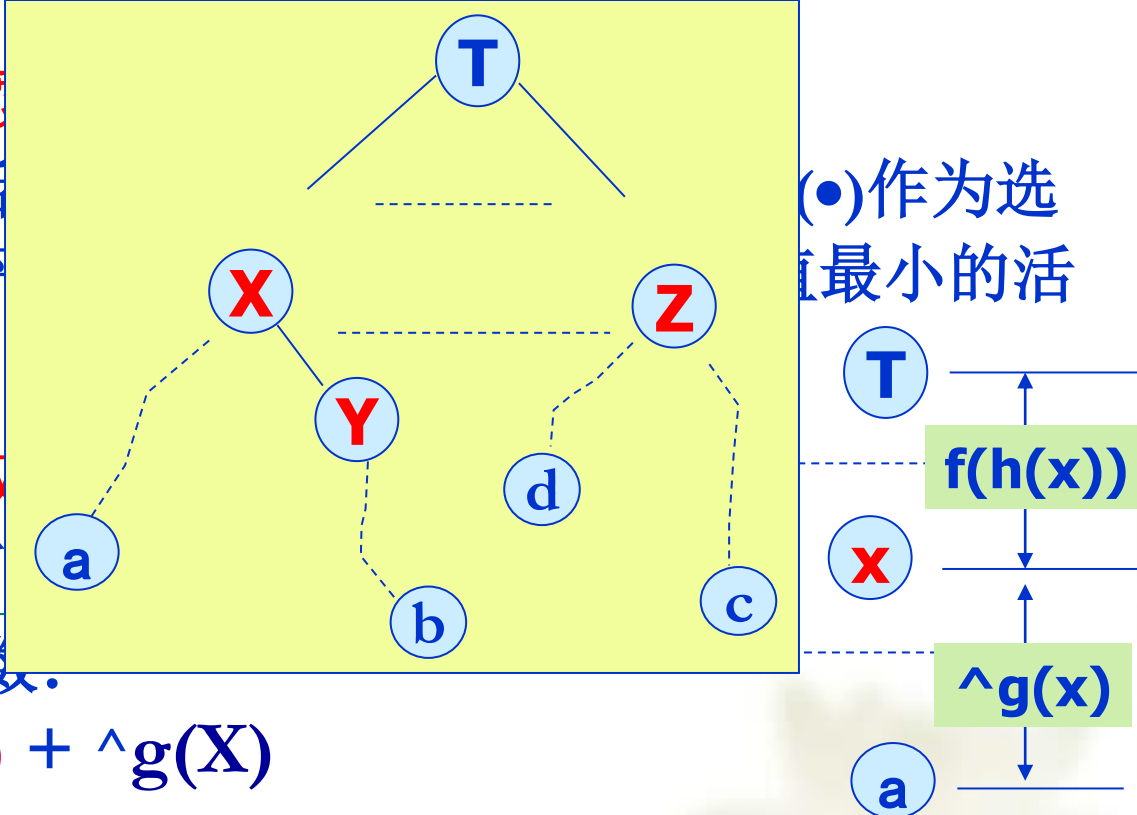
➤ 如果 $\hat{g}(\bullet) \equiv 0$ ，且 $f(h(X))$ 等于 X 在树中

出宽度优先搜索特性，成为FIFO检

➤ 如果 $f(\bullet) \equiv 0$ ，则 $\hat{c}(X) = \hat{g}(X)$ ，LC-检

特性，成为D-检索。

——这种做法不恰当： $\hat{g}(X)$ 是 $g(X)$ 的一个估计值，过分向纵
深搜索有时并不能更快接近答案结点，甚至会偏离答案结点。



(•)作为选
最小的活

——为了不使算法过分
偏向纵深搜索，引入函
数 $f(\bullet)$ 是十分必要的。

例：9宫格问题

问题描述： 在一个分成9格的方形棋盘上，放有8块编了号码的牌（见下图）。对这些牌给定一种初始排列如图(a)，要求通过一系列的合法移动将这一初始排列转换成图(b)所示的那样的目标排列。

2	8	3
1	6	4
7		5

a

1	2	3
8		4
7	6	5

b

图(a)所示的初始排列有3种可能的移动，可以将编号为5,6,7的任何一块牌移到空格。

棋盘上这些牌有9!
种不同的排列。

- 为了实现这一检索，可以将此状态空间构造成一棵树。在这棵树中，每一个结点的儿子表示由状态X通过一次合法的移动可到达的状态。

例：9 宫格问题

- ❖ 按照**FIFO**检索方法不管开始格局如何，总是采取由根开始的那条最左路径，因而检索是呆板而盲目的。
- ❖ 我们所期望的是：
 - ✧ 有一个具有一定“智能”的检索方法。这就需给状态空间树的每个结点 **X** 赋予一定的成本值 **$c(X)$** ，可将由根出发到最近目标结点路径上的每个结点 **X** 赋予这条路径长度作为他们的成本值。

- ❖ 给出一个便于计算成本估计值的函数：

$$\hat{c}(X) = f(X) + \hat{g}(X)$$

其中： **$f(X)$** 是由根到结点 **X** 的路径长度——即由初始状态到 **X** 的移动次数

$\hat{g}(X)$ 是以 **X** 为根的子树中由 **X** 到目标状态的一条最短路径长度的估计值——即还未到位的数字方块的移动次数

- ❖ 因此， **$\hat{g}(X)$** 至少应是能把状态 **X** 转换成目标状态所需的最小移动数。

❖ 所以可以这样给出最小代价函数： 初始状态为根结点。

$\wedge c(x)$ = 由初始状态到 x 的移动次数 + 还未到位的数字方块的移动次数

表示实际耗费的代价

表示至少还要移动的次数

❖ 使用 $\wedge c(X)$ 图(a)的LC-检索将结点1作为E-结点的开始。结点1在生成它的儿子结点2,3,4之后死去。变成E-结点的下一个结点是具有最小 $\wedge c(X)$ 的活结点。

2	8	3
1	6	4
7		5

a

不可走回头
右下左上

$$\wedge c(1)=4$$

①

2	8	3
1	6	4
7		5

1	2	3
1	6	4
7	6	5

扩展2, 3, 4后1成为死结点,
对活结点按照最小代价排序

②

2	8	3
1	6	4
	7	5

③

2	8	3
1		4
7	6	5

④

2	8	3
1	6	4
7	5	

$L=(3,2,4)$

⑤

2	8	3
	1	4
7	6	5

⑥

2		3
1	8	4
7	6	5

⑦

2	8	3
1	4	
7	6	5

$L=(5,6,2,4,7)$

⑧

	8	3
2	1	4
7	6	5

⑨

2	8	3
7	1	4
	6	5

⑩

	2	3
1	8	4
7	6	5

⑪

2	3	
1	8	4
7	6	5

$L=(6,2,4,7,8,9)$

$L=(10,2,4,7,8,9,11)$

⑫

1	2	3
	8	4
7	6	5

$L=(12,2,4,7,8,9,11)$

⑬

1	2	3
8		4
7	6	5

还有一个, 不需要再扩展了

$L=(2,4,7,8,9,11)$

$$\wedge c(x)=c(13)=5$$

找到一个解

9.1.3 LC-检索的抽象化控制

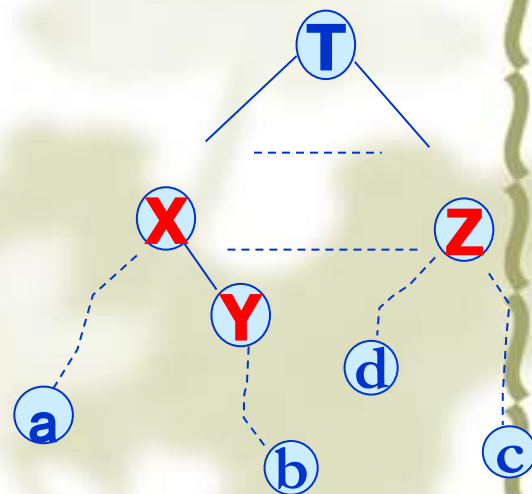
- ❖ 设 T 是一棵状态空间树， $C(.)$ 是 T 中结点的成本函数。如果， $C(X)$ 是其根为 X 的子树中任一答案结点的最小成本，则 $C(T)$ 是 T 中最小成本答案结点的成本。
- ❖ 由于函数 $C(.)$ 不容易实现，所以使用一个对 $C(.)$ 估值的启发性函数 $\hat{C}(.)$ 来代替。
- ❖ 这个启发函数应易于计算并具有如下性质：如果 X 是一个答案结点或者是一个叶结点，则 $C(X) = \hat{C}(X)$ 。

ADD(X): 将满足约束的新 X 放入活结点表中。

LEAST(X):

将一个活结点(具有最小估计值的 $\hat{c}$)作为 E ，并从活结点表中删除，放入变量 X 中返回。

为获取路径，用**PARENT**信息段将 X 与父结点相连。 ——算法中使用



算法9.1 LC-检索

Line procedure LC($T, \wedge c$) // 找一个答案结点进行检索

if T 是答案结点 then 输出 T ; return end if

$E \leftarrow T$ // E -结点 //

将活结点表初始化为空

loop

for E 的每个儿子 X do

对新的 E
扩展其儿子

if X 是答案结点 then 输出从 X 到 T 的那条路径
return, end if

call **ADD**(X) // 满足约束的 X 是新的活结点 //

PARENT(X) $\leftarrow E$. . .

保存结点 X 的父结点 E

repeat

if 不再有活结点 then print('no answer node')
stop; end if

call **LEAST**(E) . . .

将一个活结点(具有
最小估计值的)作为 E ,
并从队列中删除

repeat

End LC

将一个活结点加入到队列中

- 以上讨论，LC只有在找到一个答案结点或者在生成并检索了这棵状态空间树时才会终止。
- 对于没有答案结点的无限状态空间树，LC不会终止。
- LC检索、宽度优先检索算法(BFS)、D-检索算法基本相同。

唯一不同之处在于活结点表的构造上，即仅在于得到下一个E-结点所使用的选择规则不同。

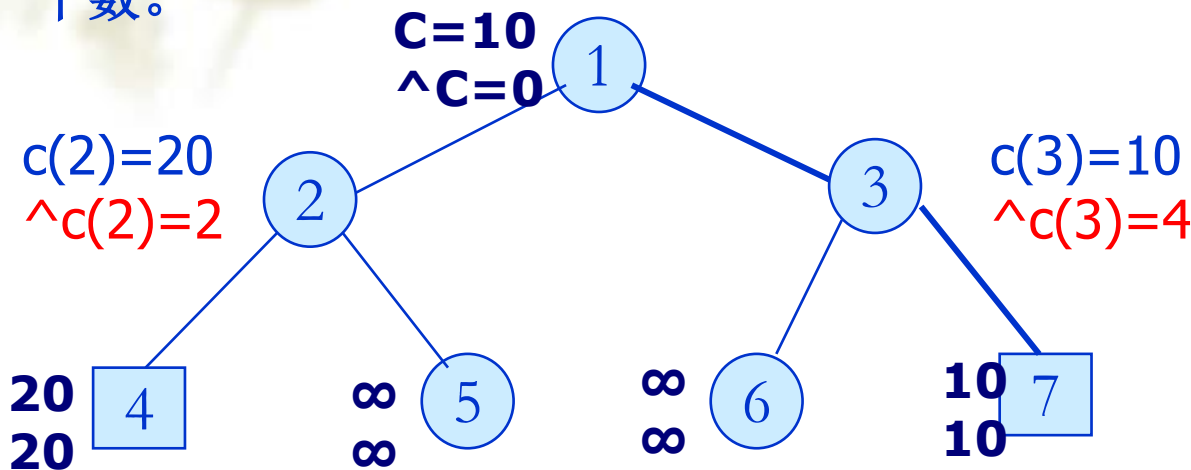
- ◆ 如果活结点表作为队列来实现，用LEASL(X)和ADD(X)算法从队列中删去或加入元素，则LC就转换成FIFO检索。
- ◆ 如果活结点表作为一个栈来实现，用LEASL(X)和ADD(X)算法从栈中删去或加入元素，则LC就转换成LIFO检索。
- ◆ LC检索也常用min堆作为活结点表。

如：九宫格的处理

9.1.4 LC-检索的特性

在许多应用中，希望在所有的答案结点中找到一个最小成本的答案结点。

- ❖ LC是否一定能找到具有最小成本 $c(G)$ 的答案结点 G 呢？回答是否定的。
- ❖ 考虑下图所示的状态空间树，方形叶子结点为答案结点。每个结点有两个数。



$c(2) > c(3)$ 但 $^c(2) \leq ^c(3)$, 在活结点表中会先选择 2 扩展, 导致不能产生最小成本的答案。

反之：如果使每一对 $c(x) < c(y)$ 的 x, y 结点都有 $^c(x) \leq ^c(y)$, 那么, LC 总会找到一个最小成本答案结点。

改进算法, 对于结点 X , 有 $^c(X) \leq c(X)$, 对答案结点 X , 有 $^c(X) = c(X)$ 。

LC 没能达到这个最小成本答案结点的原因在于, 有两个这样的结点 x 和 y , 当 $c(x) > c(y)$ 时, 却 $^c(x) \leq ^c(y)$, 此时沿 x 不能找到最小成本的答案结点。

算法9.2 找最小成本答案结点的LC-检索

Line procedure LC1(T, ^c)

$E \leftarrow T$

置活结点表为空

loop

if E是答案结点 且 $^c(E)=c(E)$

then 输出从E到T的路径 ; return end if

for E的每一个儿子X do

call ADD(X); PARENT(X) $\leftarrow$ E

repeat

if 不再有活结点 then print ('No answer node')

stop

end if

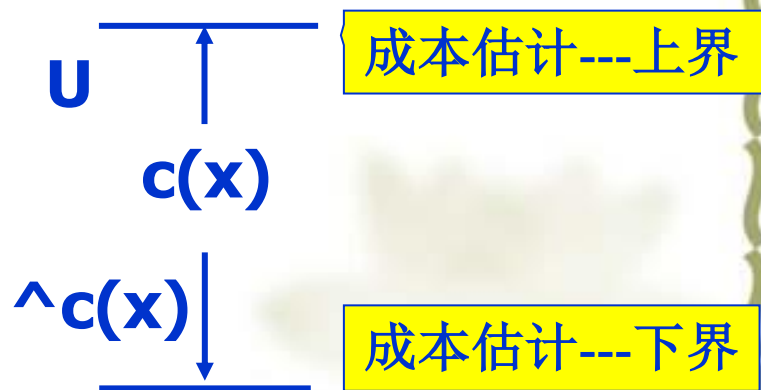
call LEAST(E)

repeat

End LC1

减少了盲目性

- **好处：**采用下界函数使算法具有一定的智能，减少了盲目性。
- **问题：**由于估值给出的不准，总会有 $\hat{c}(x) < c(x)$ ，对活结点表中的结点要逐一搜索，导致此时的搜索与D-搜索没什么两样，甚至变成穷举法。
- **解决方法：**利用上界剪掉一些子树(活结点)，使搜索加速。



构造剪枝函数：

分枝限界方法中的内容：

- 1) 最小代价搜索，是在先广后深的基础上，提出如何产生下一个活结点问题。
- 2) 再一个是限界问题，利用上界去掉活结点表中的一些结点，使活结点表中的减少，提高搜索效率。

9.2 分枝-限界算法

- ❖ 检索状态空间树的各种分枝-限界方法都是在生成当前E-结点的所有儿子之后再将其另一个结点变成E-结点。
- ❖ 假定每一个答案结点X有一个与其相联系的 $C(X)$ ，并且假定会找到最小成本的答案结点。
- ❖ 使用一个使得 $\hat{C}(X) \leq C(X)$ 的成本估计函数 $\hat{C}(\cdot)$ 来给出可由任一结点X得出的解的下界。
- ❖ 采用下界函数使算法具有一定的智能，减少了盲目性。另外还可以通过设置最小成本的上界使算法进一步加速。

两种策略：

- 1) 从活结点表中选择活结点扩展的，形成一种分枝；
- 2) 用限界剪去活结点表中的以某些活结点为子树根的子树。

这个加速是通过杀死某些活结点来实现的。由于找不到 $\hat{c}(x)=c(x)$ ，避免过多的搜索。

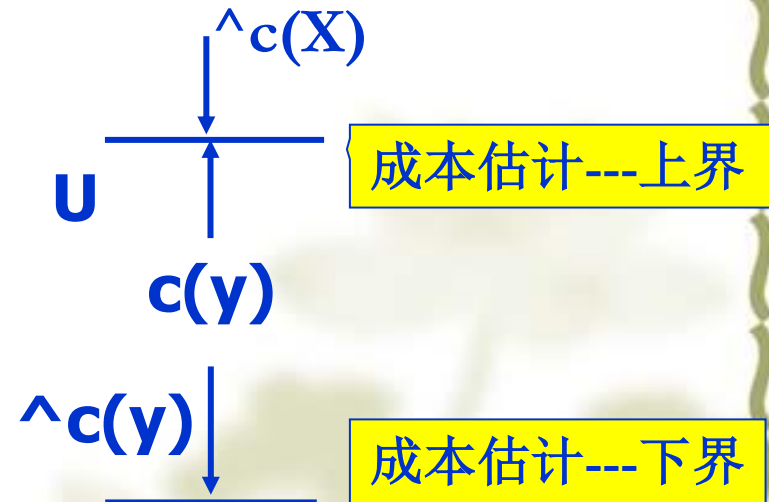
9.2 分枝-限界算法

无法找到 $\wedge c(x)=c(x)$ 的结点

- ❖ 如果 U 是最小成本解的上界，则
具有 $\wedge C(X) > U$ 的所有活结点 X 可以被杀死，
这是因为由 X 可以到达的所有答案结点有

$$U < \wedge C(X) \leq C(X)。$$

- 在已经到达一个具有成本 U 的答案结点的情况下，那些有 $U \leq \wedge C(X)$ 的所有活结点都可以被杀死。



- U 的初始值可以用某种启发性方法得到，也可以置为 ∞ ，每当找到一个新的答案结点就可以修改 U 的值。

不可走回头
右下左上

①

2	8	3
1	6	4
7		5

④

$L=(1)$

代价5

所有在2, 3, 4, 5级上的剩余结点的代价应分别大于4, 3, 2和1。

右 下 左

② ③ ④

2	8	3
1	6	4
	7	5

2	8	3
1		4
7	6	5

2	8	3
1	6	4
7	5	

$L=(3,2,4)$

右 下 左

⑤ ⑥ ⑦

2	8	3
	1	4
7	6	5

2		3
1	8	4
7	6	5

2	8	3
1	4	
7	6	5

$L=(5,6,2,4,7)$

下 上 右 左

⑧ ⑨ ⑩ ⑪

	8	3
2	1	4
7	6	5

2	8	3
7	1	4
	6	5

	2	3
1	8	4
7	6	5

2	3	
1	8	4
7	6	5

$L=(6,2,4,7,8,9)$

$L=(10,2,4,7,8,9,11)$

⑫

1	2	3
	8	4
7	6	5

⑤

$L=(12,2,4,7,8,9,11)$

⑬

1	2	3
8		4
7	6	5

⑤

$L=(2,4,7,8,9,11)$

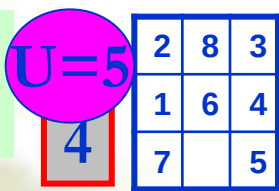
$\wedge c(x)=c(13)=5$

找到一个解

还有一个，不需要再扩展了

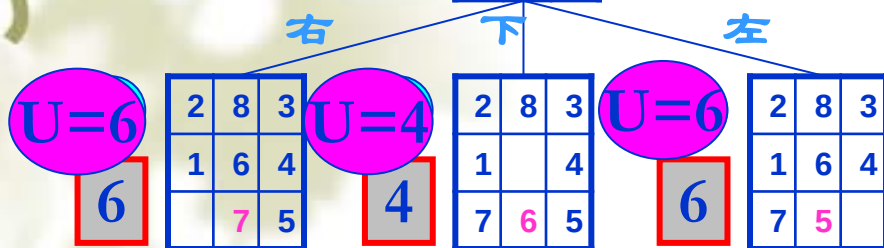
移动步数多的就舍弃

不可走回头
右下左上



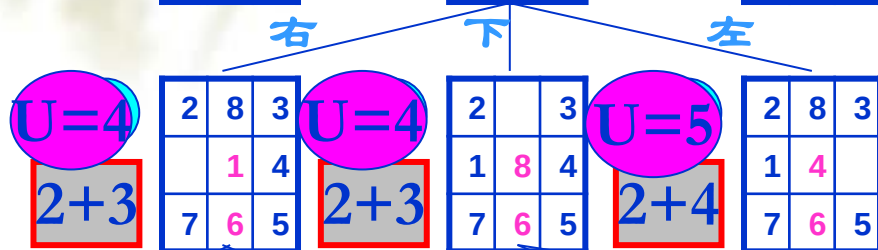
L=(1)

代价5

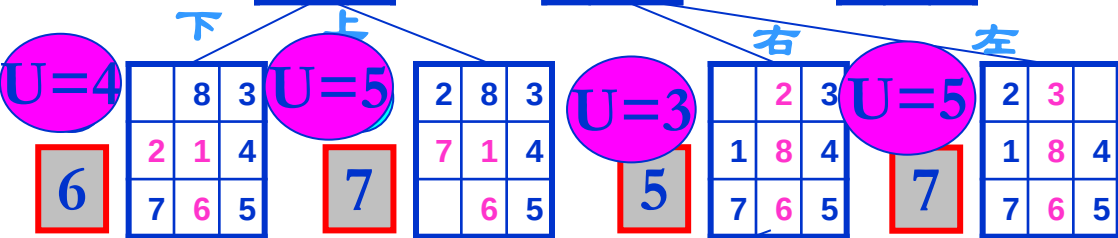


L=(3,2,4)

所有在2, 3, 4, 5级上的剩余结点的代价应分别大于4, 3, 2和1。

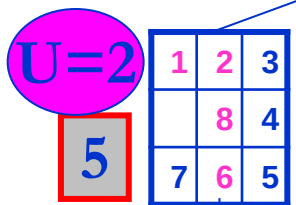


L=(5,6,2,4,7)



L=(6,2,4,7,8,9)

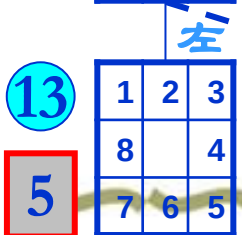
L=(10,2,4,7,8,9,11)



L=(12,2,4,7,8,9,11)

$\wedge c(x)=c(13)=5$

找到一个解



L=(2,4,7,8,9,11)

还有一个, 不需要再扩展了

移动步数多的就舍弃

- 如何根据上述思想来求解最优化问题的分枝-限界算法？
- 以下只考虑极小化问题。(极大化问题可以通过改变目标函数的符号很容易转化成极小化问题)
- 假定目标函数取最小值时为最优解。为了找到最优解，需要将最优解的检索表示成对状态空间树答案结点的检索，答案结点的成本 $c(x)$ 是所有结点成本的最小值。
- 最简单的方法是直接把目标函数作为成本函数 $c(\cdot)$ 。

使用代价函数的概念处理最优化问题。
——与目标函数有关

此时

可行解的结点 x 的 $C(x)$ =目标函数值

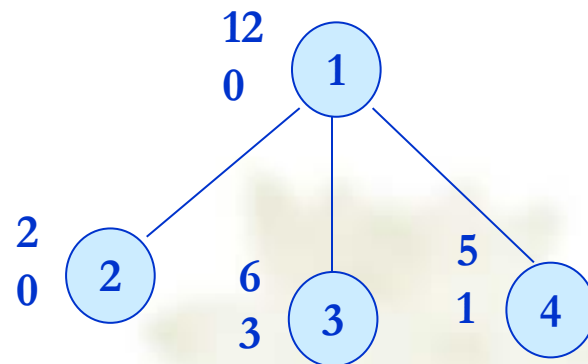
不可行解的结点 x 的 $C(x) = \infty$

耗费-最小，得到-最大
利润-最大，损失-最小

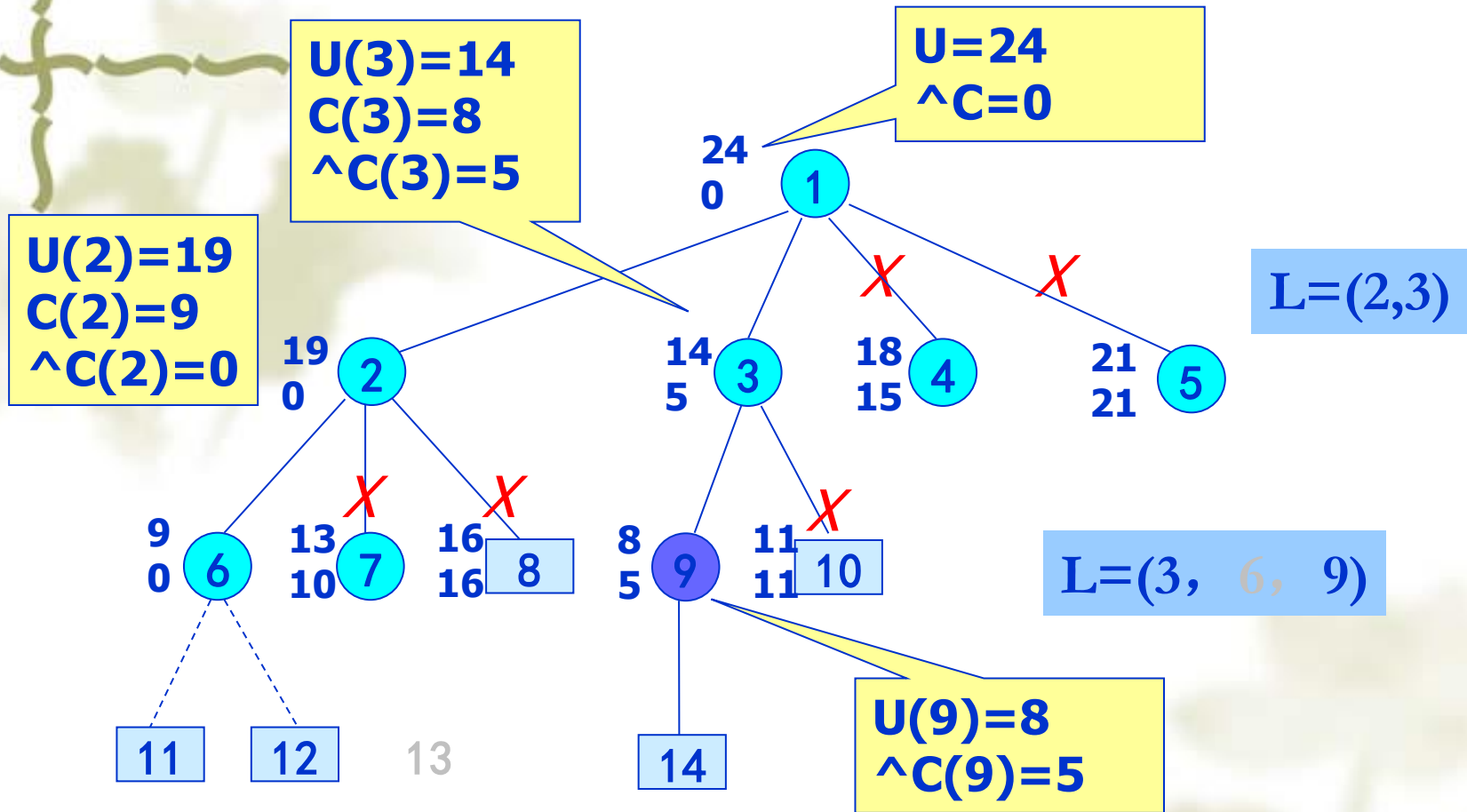
——不是对计算的估计，是对目标函数的估计

这时算法需要一个上界U，U是该子树上最小代价答案结点代价的上界值。执行过程中要记录已知的最小代价的上界，也即最小代价答案结点的代价值不会超过U。
(超过的会剪掉了,U值是在不断修改的)

U(2)=2,表示可以找到代价不超过**2**的解，**U=U(2)**。而结点**3**找到的解最小代价至少有**3**，所以活结点**3**可以删去。



$$U(2) < c(3) \leq c(3)$$



例：带限期的作业排序问题。将此问题一般化，允许作业有不同的处理时间。假定有 n 个作业和一台处理机，每个作业 i 与一个三元组 (p_i, t_i, d_i) 相联系，它要求 t_i 个单位处理时间，如果在期限 d_i 内完成可获利润 p_i 。问题的目标是从这 n 个作业中选取一个子集合 J ，要求在 J 中的作业都能在相应的期限内完成且使在 J 中的作业利润值和最大。这样的 J 就是最优解。

实例： $n=4, (p_1, t_1, d_1)=(5, 1, 1); (p_2, t_2, d_2)=(10, 2, 3); (p_3, t_3, d_3)=(6, 1, 2); (p_4, t_4, d_4)=(3, 1, 1)$ 。

分析：类似子集和数问题。采用可变大小元组或固定大小元组，这里用可变大小元组 $(x_1, x_2, \dots, x_k)$ 表示解， x_i 为作业号。

显约束: $x_i \in \{0, 1, \dots, n-1\}$ 且 $x_i \leq x_{i+1} (0 \leq i < n-1)$;

隐约束: 对于选入子集J的作业 ($x_1, x_2, \dots, x_k$), 存在一种作业排列, 使J中作业均能在截止期限完成。

目标函数: 作业子集J中所有作业所获取的收益之和为 $\sum_{i \in J}^n p_i$

使得总收益最大的作业子集是问题的最优解。

如果用最小值来表达最优解, 则可改变目标函数。

分析: 如果全部作业都可安排, 可获利为 $\Sigma p(i)$;

如果作业i没能选入J中, 则损失利润 $p(i)$.

全部作业
都安排产
生的利润

设S是没有选入J中的作业集, 此时未入选子集J的作业所导致的损失是:

$$\sum_{i \in S} p(i) = \sum_{i \notin J} p(i) = \sum_{i=1}^n p(i) - \sum_{i \in J} p(i)$$

可以选入作业
子集J中利润

显约束: $x_i \in \{0, 1, \dots, n-1\}$ 且 $x_i \leq x_{i+1} (0 \leq i < n-1)$;

隐约束: 对于选入子集J的作业 ($x_1, x_2, \dots, x_k$), 存在一种作业排列, 使J中作业均能在截止期限完成。

目标

即不在J中的作业利润值和最小。

为 $\sum_{i \in J}^n p_i$

如果用最小值求解, 则可改变目标函数。

分析: 如果全部作业都可安排, 可获利为 $\sum p(i)$;

如果作业不能选入J中, 则损失利润 $p(i)$.

全部作业
都安排产生
的利润

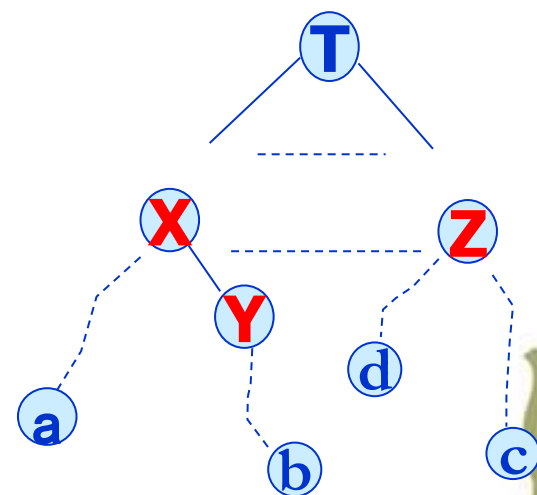
设S是没有选入J中的作业集, 此时未入选子集J的作业所导致的损失是:

$$\sum_{i \in S} p(i) = \sum_{i \notin J} p(i) = \sum_{i=1}^n p(i) - \sum_{i \in J} p(i)$$

可以选入作业
子集J中利润

这里考虑用上下界函数来求解

设 x 是状态空间树上的一个结点。作业 J 中 $x(1), x(2) \dots x(k)$ 是已入选的作业子集，它代表一条从根到 x 的路径。



如果用 $x(i)$ 表示第 i 步选择的作业号，例如， $x(1)=1$ 表示第一步选择了第1个作业，没有舍弃任何作业，这种选择没有任何利润损失；如果 $x(1)=3$ 表示第一步选择了第3个作业，舍弃了作业1，2，损失的利润是 $p(1)+p(2)$ 。

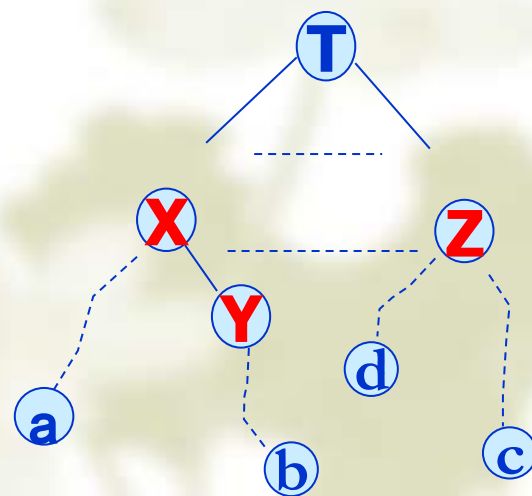
对于结点 x ，已经损失了的利润和作为 $\hat{c}(x)$ ——而最大的损失，即获得最小的利润。

因此到达答案结点时损失的利润不会低于它：

$$U(x) \text{ 是 } \sum_{i=1}^n p(i) - \sum_{i \in J} p(i)$$

➤ 下界的产生：设 n 个作业，都不扔掉，主观上按顺序选择就是不扔掉，因此，第1个作业选择后产生效益 p_1 ，接着产生 p_2 ，...这样利润可能损失为0，即利润损失的下界可能为0，用已经损失的利润和作为下界 $\wedge c(x)$ ，事实上的利润损失会大于0(由于期限的原因有些作业不能加入，使得利润损失增加，就会大于0)。

因此下界用已经损失的利润和给出，考虑约束后，有不能放入的作业，损失的利润就会产生，就出现大于0。



结点X的下界函数值为:

$$\hat{c}(X) = \sum_{i \notin J, i < x_k} p(i)$$

已经损失的利润之和

考虑：不在J中的作业利润值和最小。

表达此时未能入选J中的作业所造成的损失。它是最终损失的下界，即 $\hat{c}(X) \leq c(X)$ 。

(最好情况是0)

➤ 上界的产生：已经加入了部分作业，还有部分未加入的作业，则可能损失的最大利润是那些没能放进J中的作业利润值和。由于可能还会有作业选入，最大利润损失还会减少，所以上界 $U(x)$ ，有

$$\hat{c}(x) \leq c(x) \leq U(x)$$

可以看出给出的 $\hat{c}(x)$ 和 U 是合理的。

所以 $U(x)$ 是在选择 x 下，还没有得到的利润之和，即还没有选入J中的利润之和。

结点X的上界函数定义为:

$$U(X) = \sum_{i \notin J, i=1,2,\dots,n} p_i = \sum_{i \notin J, i < x_k} p_i + \sum_{i=(x_k+1),\dots,n} p_i$$

U(X)由两部分组成:



(1) 在 x_k 之前已知未入选J中的作业所造成的损失;

(2) 对作业 x_k 以后所有作业都未入选可能造成的损失。

显然, 此值是在X处可以预计的最大损失, 必有 $c(X) \leq U(X)$ 。

所以上下界函数为: $\hat{c}(X) \leq c(X) \leq U(X)$ 。

例如: $x(1)=1$ 表示第一步选择了第1个作业, 没有舍弃任何作业, 即没有任何利润损失; 但可能损失的是作业2,3,4的利润。

如果 $x(1)=3$ 表示第一步选择了第3个作业, 舍弃了作业1和2, 损失的利润是 $p(1)+p(2)$ 。可能损失的是作业4的利润。



上界函数定义为:

$$U(X) = \sum_{i \notin J, i=1,2,\dots,n} p_i = \sum_{i \notin J, i < x_k} p_i + \sum_{i=(x_k+1),\dots,n} p_i$$

U(X)由两部分组成:

p1

●

p2

●

p3

●

.....

●

可能损失的最大利润

$\wedge c(x) \leq c(x) \leq U(x)$

- (1) 在 x_k 之前已知未入选J中的作业所造成的损失;
- (2) 对作业 x_k 以后所有作业都未入选可能造成的损失。

显然, 此值是在X处可以预计的最大损失, 必有 $c(X) \leq U(X)$ 。
 所以上下界函数为: $\wedge c(X) \leq c(X) \leq U(X)$ 。

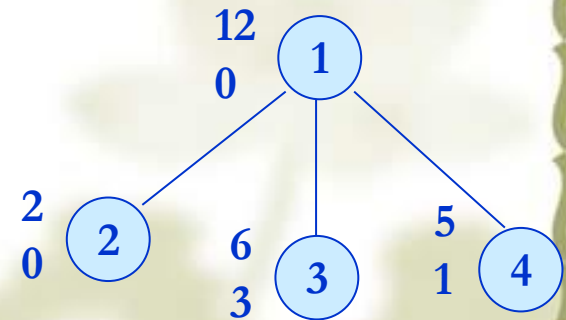
例如: $x(1)=1$ 表示第一步选择了第1个作业, 没有舍弃任何作业, 即没有任何利润损失; 但可能损失的是作业2,3,4的利润。

如果 $x(1)=3$ 表示第一步选择了第3个作业, 舍弃了作业1和2, 损失的利润是 $p(1)+p(2)$ 。可能损失的是作业4的利润。

初始时 $U = \infty$ 或 $U = \sum_{i=1}^n p_i$ 。 (所有的利润都被损失)

对于新产生的活结点 x ，如果 $U(x) < U$ (U 是当前的上限值)，则用 $U(x)$ 替换 U 。对于活结点中的结点 y ，有 $\hat{c}(y) > U$ ，则 y 可以从活结点表中删去。

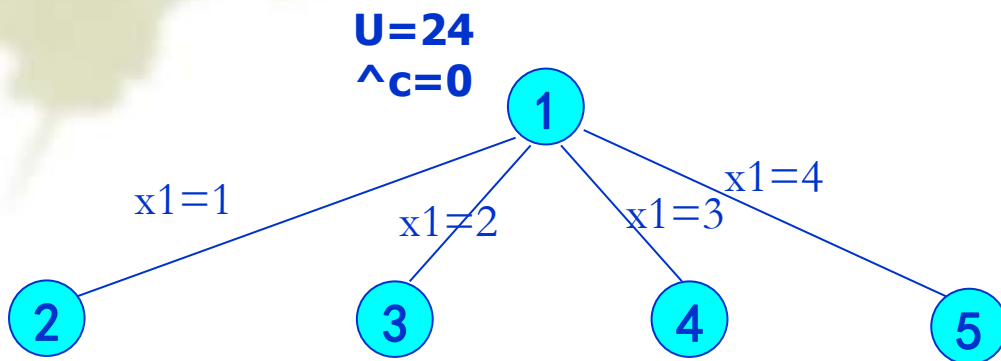
$U(2) = 2$ ，表示可以找到代价不超过2的解， $U = U(2)$ 。而结点3找到的解最小代价至少有3，所以活结点3可以删去。



$$U(2) \leq \hat{c}(3)$$



实例： $n=4, (p_1, t_1, d_1)=(5, 1, 1); (p_2, t_2, d_2)=(10, 2, 3);$
 $(p_3, t_3, d_3)=(6, 1, 2); (p_4, t_4, d_4)=(3, 1, 1)$ 。此实例的解空间
 由作业指标集 $(1, 2, 3, 4)$ 的所有可能的子集组成。



$$^c(X) = \sum_{i \notin J, i < x_k} p_i$$

$$U(X) = \sum_{i \notin J, i=1,2,\dots,n} p_i = \sum_{i \notin J, i < x_k} p_i + \sum_{i=(x_k+1),\dots,n} p_i$$

目的：损失最小，即利润最大

已经
损失的

还可能
有的损
失

对4个作业进行选择:

➤ **选择J1:** 得到 p_1 ,最小损失为0; 此时也可能剩余的作业都不能选择, 则此时最大的利润损失是 $p_2+p_3+p_4=19$ 。

➤ **不选择J1,而选择J2:** 得到 $p_2=10$,最小损失为 $p_1=5$; 此时也可能剩余的作业 p_1 、 p_3 、 p_4 不能选择, 则此时最大的利润损失是 $p_1+p_3+p_4=14$ 。

➤ **不选择J1、J2,而选择J3:** 得到 $p_3=6$,最小损失为 $p_1+p_2=15$; 此时也可能剩余的作业 p_1 、 p_2 、 p_4 不能选择, 则此时最大的利润损失是 $p_1+p_2+p_4=18$ 。

➤ **不选择J1、J2、J3,而选择J4:** 得到 $p_4=3$,最小损失为 $p_1+p_2+p_3=21$; 此时J1、J2、J3不选择, 则最大的利润损失是 $p_1+p_2+p_3=21$ 。

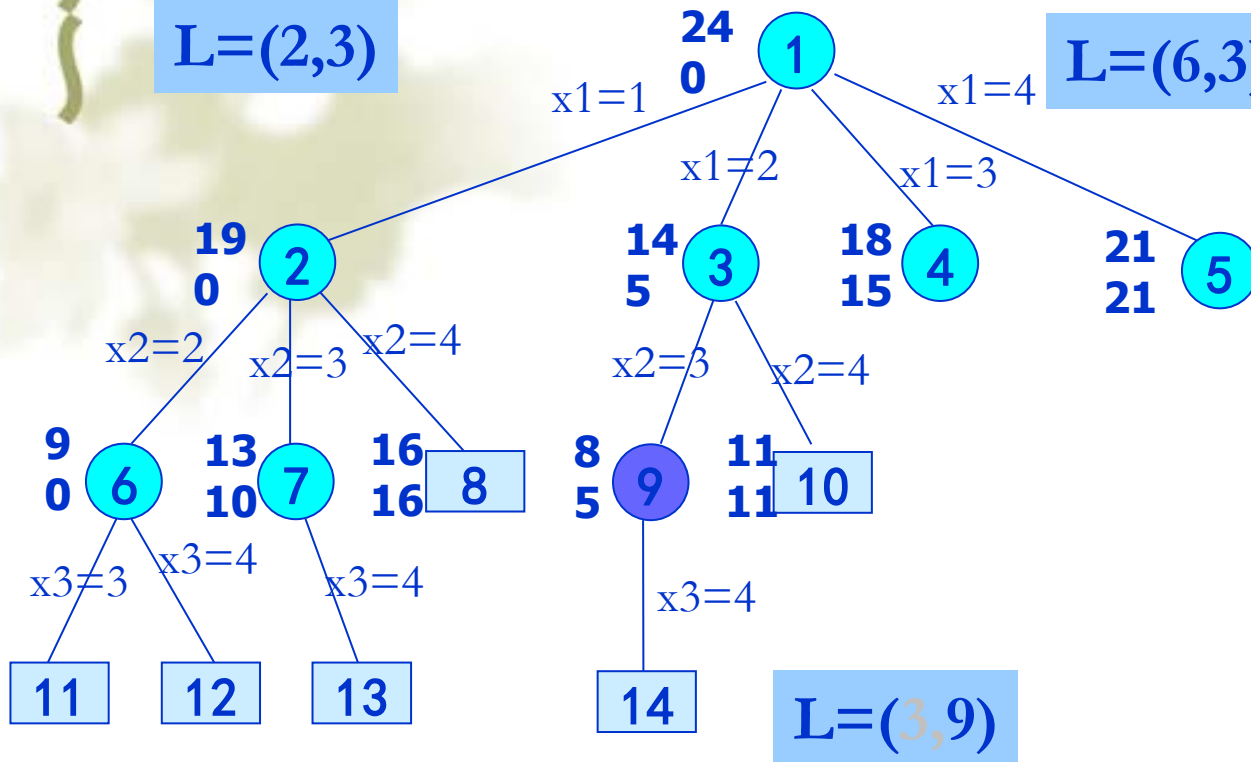
实例: $n=4$,

$(p_1, t_1, d_1) = (5, 1, 1);$
 $(p_2, t_2, d_2) = (10, 2, 3);$
 $(p_3, t_3, d_3) = (6, 1, 2);$
 $(p_4, t_4, d_4) = (3, 1, 1).$
解空间由作业指标集 $(1, 2, 3, 4)$ 的所有可能的子集组成。

1, 5	19 0
2, 10	14 5
3, 6	18 15
4, 3	21 21

$L=(2,3)$

$L=(6,3)$



实例: $n=4$,
 $(p_1, t_1, d_1)=(5, 1, 1)$;
 $(p_2, t_2, d_2)=(10, 2, 3)$;
 $(p_3, t_3, d_3)=(6, 1, 2)$;
 $(p_4, t_4, d_4)=(3, 1, 1)$.
 此实例的解空间由作业指标集**(1,2,3,4)**的所有可能的子集组成.

1: 5	19 0
2: 10	14 5
3: 6	18 15
4: 3	21 21

用已经损失的利润和作为下界 $\hat{c}(x)$,

上界 $U(x)$: 可能损失的最大利润是那些没能放进 J 中的作业利润值和。

$L=(2,3)$

$L=(6,3)$

实例: $n=4$,

$(p_1, t_1, d_1) = (5, 1, 1);$

$(p_2, t_2, d_2) = (10, 2, 3);$

$(p_3, t_3, d_3) = (6, 1, 2);$

$(p_4, t_4, d_4) = (3, 1, 1).$

此实例的解空间由作业指标集 $(1, 2, 3, 4)$ 的所有可能的子集组成.

$U(2)=19$
 $C(2)=9$
 $\wedge C(2)=0$

$U(3)=14$
 $C(3)=8$
 $\wedge C(3)=5$

$U(9)=8$
 $\wedge C(9)=5$

$L=(3, 9)$

选择J2, 没有选择J1, 所以损失利润5, 可能的损失是J3, J4的利润和9, 加上已经损失的利润5=14

用已经损失的利润和作为下界 $\wedge c(x)$,

上界 $U(x)$: 可能损失的最大利润是那些没能放进J中的作业利润值和。

	18
3: 6	15
	21
4: 3	21

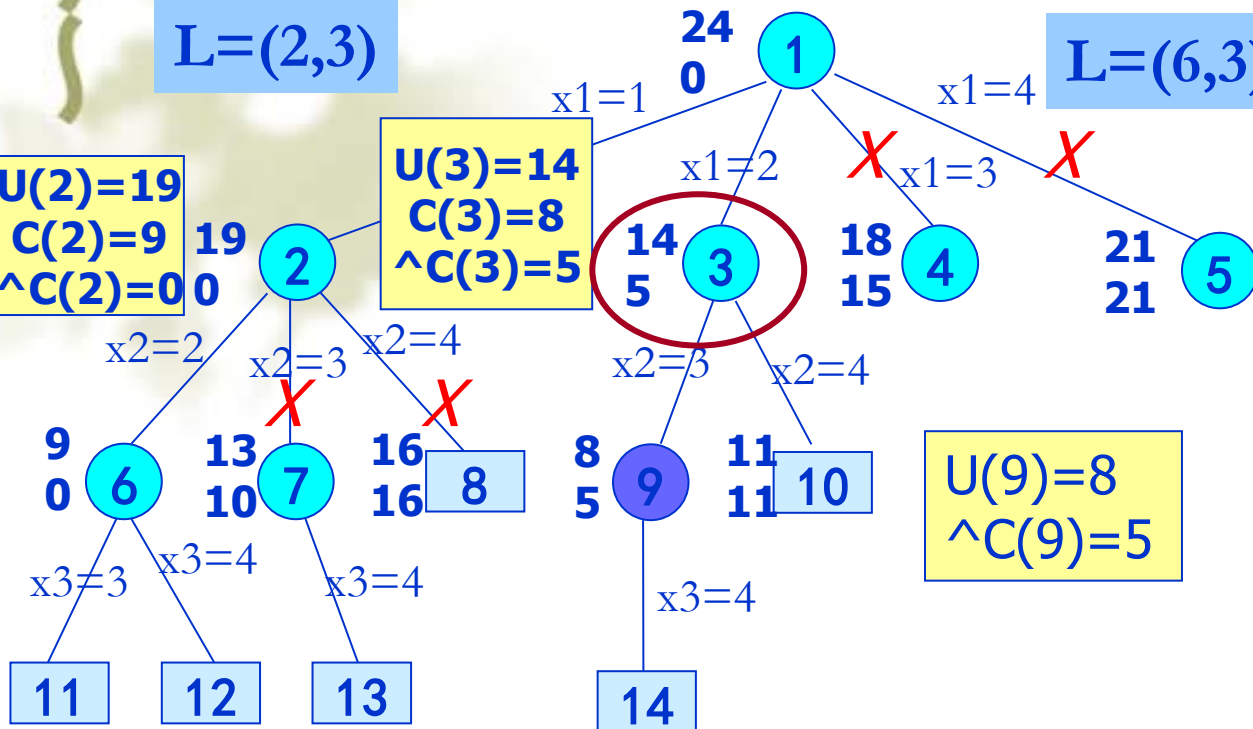
$L=(2,3)$

$U(2)=19$
 $C(2)=9$
 $\wedge C(2)=0$

$U(3)=14$
 $C(3)=8$
 $\wedge C(3)=5$

$L=(6,3)$

$U(9)=8$
 $\wedge C(9)=5$



实例: $n=4$,
 $(p_1, t_1, d_1)=(5, 1, 1)$;
 $(p_2, t_2, d_2)=(10, 2, 3)$;
 $(p_3, t_3, d_3)=(6, 1, 2)$;
 $(p_4, t_4, d_4)=(3, 1, 1)$.
 此实例的解空间由作业指标集(1,2,3,4)的所有可能的子集组成.

找到了一个利润损失不超过14的解, 而结点4和5的最小利润损失分别为15, 21, 已经超过了结点3的 $U(3)$, 所以不需要再搜索结点4和5, 即活结点表中只有结点2和3. 对结点2扩展后产生结点6和7.

用已经损失的利润和作为下界 $\wedge c(x)$,

上界 $U(x)$: 可能损失的最大利润是那些没能放进 J 中的作业利润值和.

1: 5	19 0
2: 10	14 5
3: 6	18 15
4: 3	21 21

$L=(2,3)$

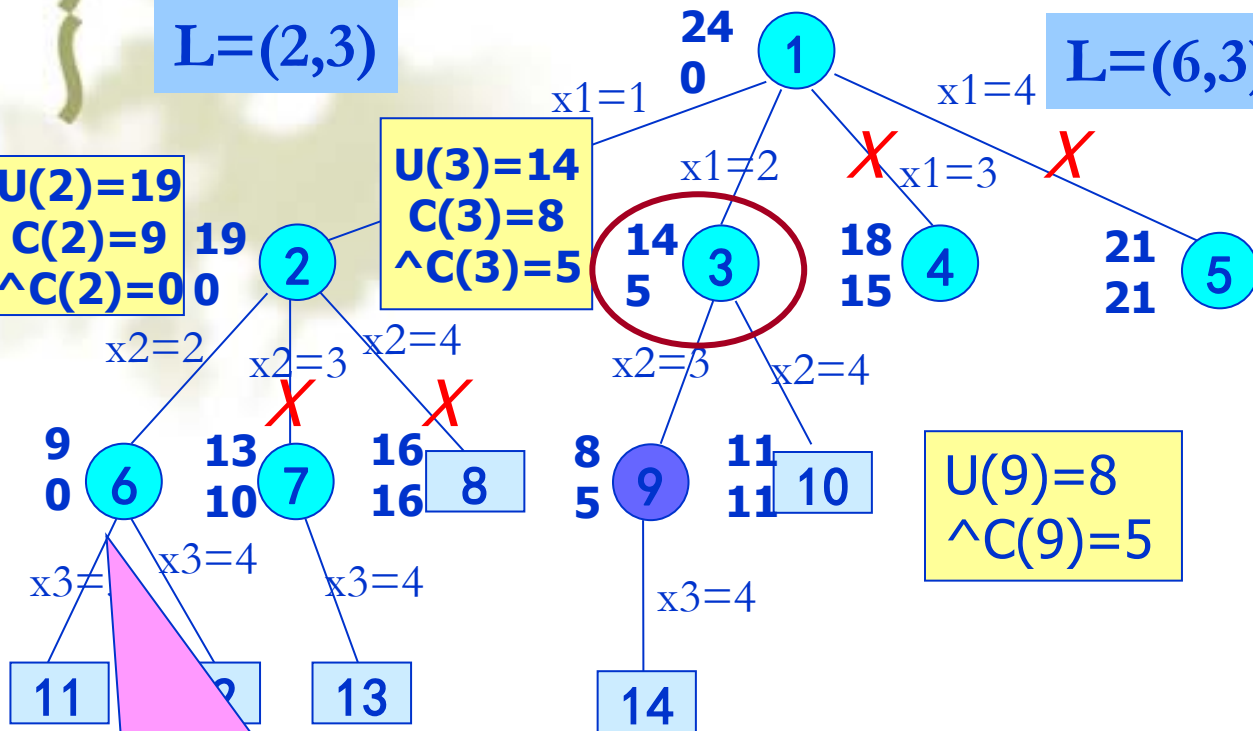
$L=(6,3)$

$U(2)=19$
 $C(2)=9$
 $\wedge C(2)=0$

$U(3)=14$
 $C(3)=8$
 $\wedge C(3)=5$

$U(9)=8$
 $\wedge C(9)=5$

实例: $n=4$,
 $(p_1, t_1, d_1)=(5, 1, 1)$;
 $(p_2, t_2, d_2)=(10, 2, 3)$;
 $(p_3, t_3, d_3)=(6, 1, 2)$;
 $(p_4, t_4, d_4)=(3, 1, 1)$.
 此实例的解空间由作业指标集(1,2,3,4)的所有可能的子集组成.



扩展⑥时, 由于期限的约束, J3不能放入J中, 而此时只放入了J1和J2, 其利润为15, 损失为 $c(6)=9$, 显然 $c(2)=9$

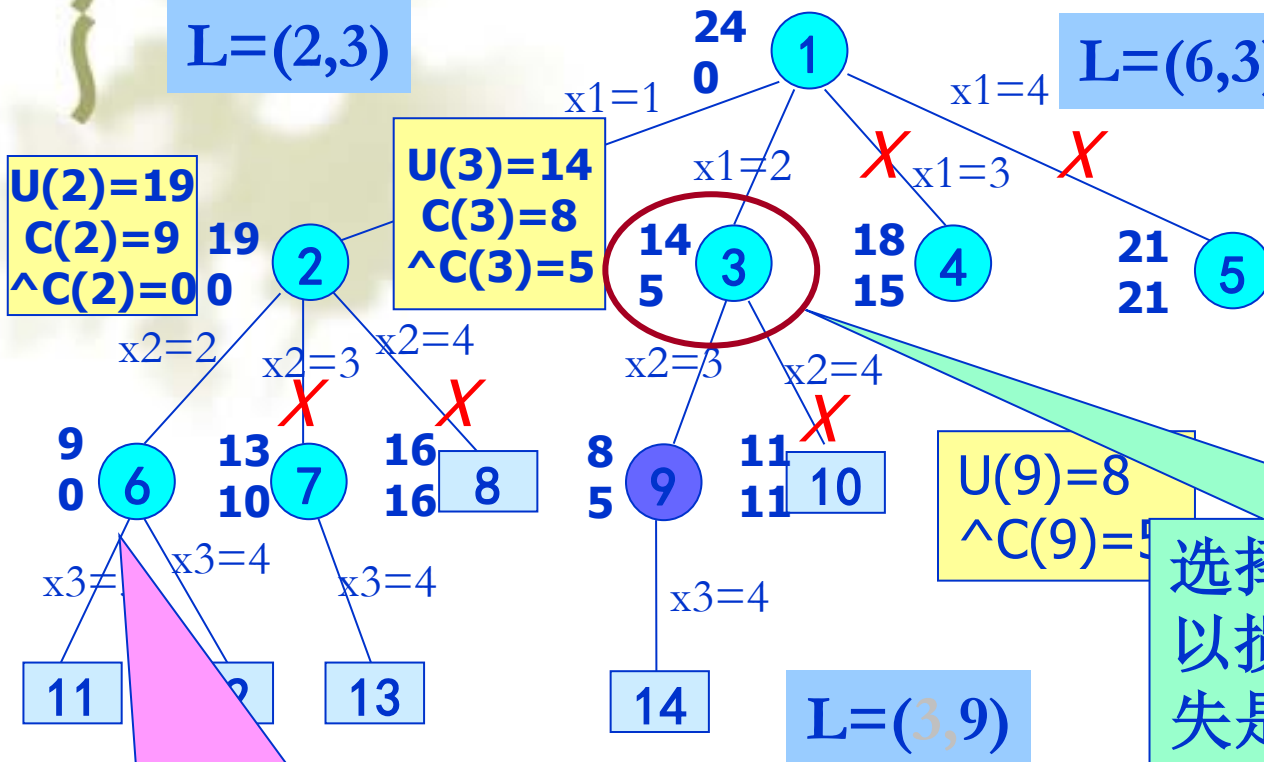
上界 $U(x)$: 可能损失的最大利润是那些没能放进J中的作业利润值和。

1: 5	19 0
2: 10	14 5
3: 6	18 15
4: 3	21 21

实例: $n=4$,
 $(p_1, t_1, d_1) = (5, 1, 1)$;
 $(p_2, t_2, d_2) = (10, 2, 3)$;
 $(p_3, t_3, d_3) = (6, 1, 2)$;
 $(p_4, t_4, d_4) = (3, 1, 1)$.
 此实例的解空间由作业指标集 $(1, 2, 3, 4)$ 的所有可能的子集组成.

$L=(2,3)$

$L=(6,3)$



选择J2,没有选择J1, 所以损失利润5,可能的损失是J3,J4的利润和9, 加上已经损失的利润5=14

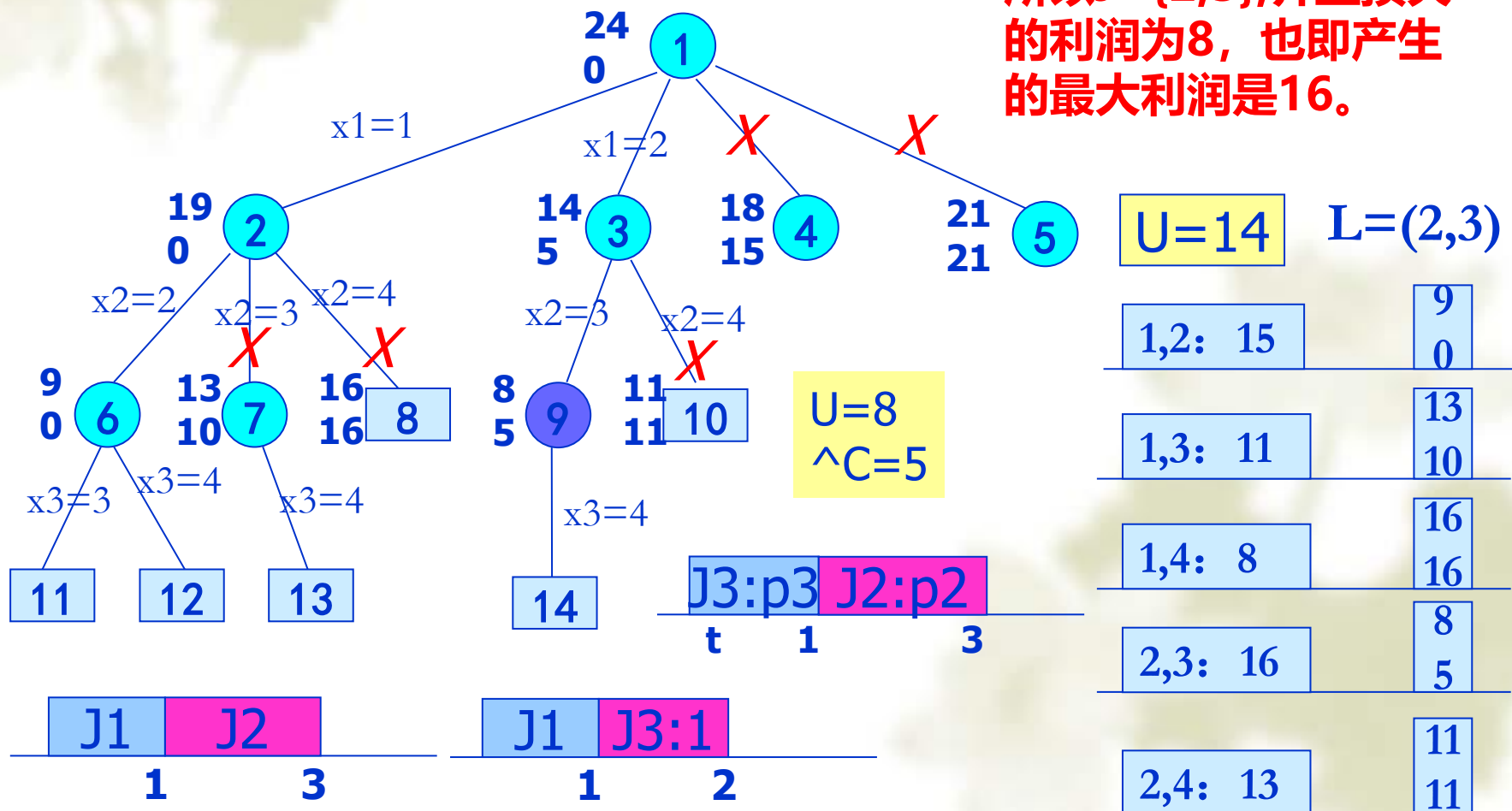
扩展⑥时, 由于期限的约束, J3不能放入J中, 而此时只放入了J1和J2, 其利润为15, 损失为 $c(6)=9$, 显然 $c(2)=9$

上界 $U(x)$: 可能损失的最大利润是那些没能放进J中的作业利润值和。

		18
3: 6		15
		21
4: 3		21

实例: $n=4, (p_1, t_1, d_1)=(5, \mathbf{1}, 1); (p_2, t_2, d_2)=(10, \mathbf{2}, 3);$
 $(p_3, t_3, d_3)=(6, \mathbf{1}, 2); (p_4, t_4, d_4)=(3, \mathbf{1}, 1)$ 。此实例的解空间
 由作业指标集 $(\mathbf{1}, \mathbf{2}, \mathbf{3}, \mathbf{4})$ 的所有可能的子集组成。

所以 $J=\{2,3\}$, 并且损失
 的利润为8, 也即产生
 的最大利润是16。



效率分析:

上下界函数选择的好坏决定极小化问题的效率

——主要原因

- (1) 对U选择一个较好的初值可以减少所生成的结点数
- (2) 扩展一些 $\hat{c}(\cdot) > U$ 的结点可以减少所生成的结点数
- (3) 对结点X, 如果 $\hat{c}_1(X) \leq \hat{c}_2(X) \leq c(X)$, 则使用 $\hat{c}_2(X)$ 可以减少生成的结点数

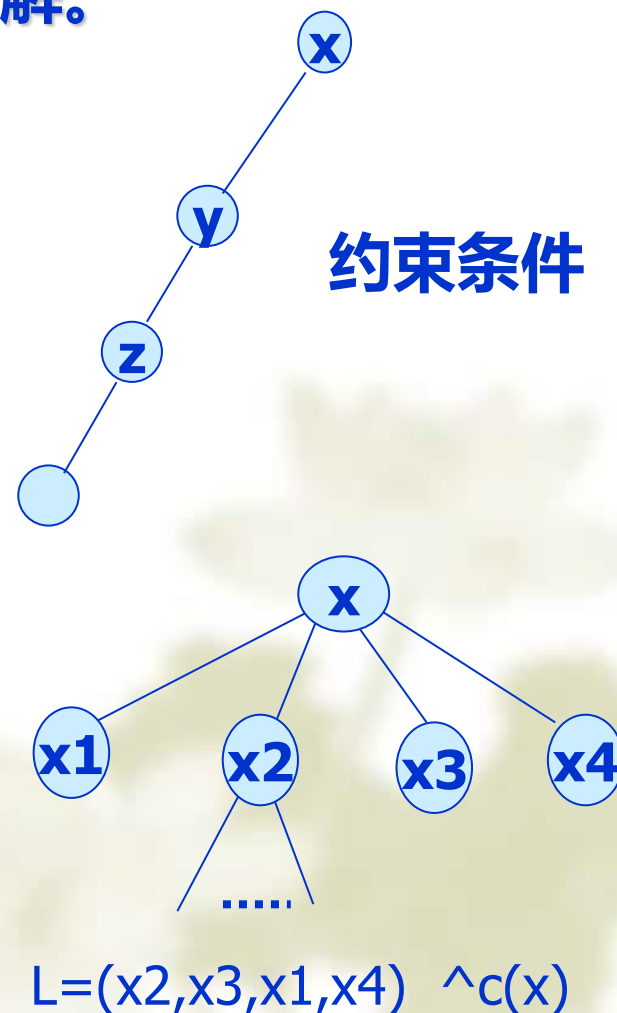
小结

分枝限界(剪枝法)与回溯法相似，也是对状态空间树进行搜索求解，以获得问题的解。

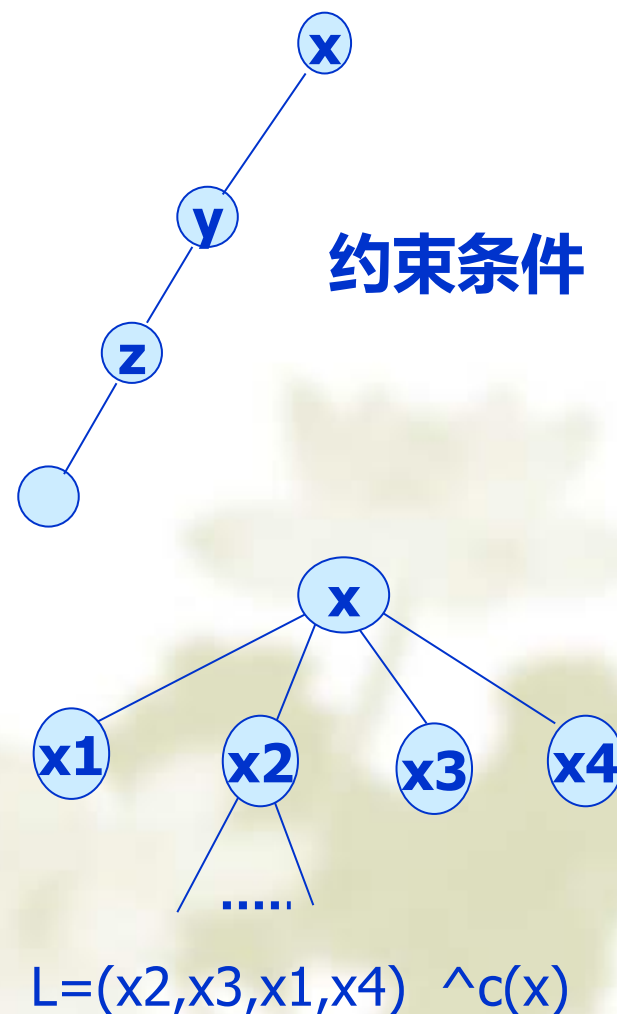
1. 分枝限界法与回溯法的不同

(1) 求解目标：回溯法的求解目标是找出解空间树中满足约束条件的**所有解**；而分枝限界法的求解目标则是找出满足约束条件的**一个解**，或是在满足约束条件的解中找出在**某种意义下的最优解**。

(2) 搜索方式的不同：回溯法以**深度优先**的方式搜索解空间树；而分枝限界法则以**先广后深或以最小耗费优先**的方式搜索解空间树。



(3) 在算法设计上，分枝限界剪枝法不仅通过**约束条件**来控制搜索路径，还通过**目标函数**的限界来减少搜索规模；分枝限界剪枝法在搜索时，将满足约束条件且不越过目标函数的子结点加入到**活结点表**中等待搜索。



2. 分枝限界法基本思想

- 分枝限界法是在生成当前扩展结点的**全部儿子**之后，再生成其他活结点的儿子，并且使用限界函数帮助避免生成不包含答案结点的子树的状态空间的检索方法。
- 在分枝限界法中，每一个**活结点**只有一次机会成为扩展结点。活结点一旦成为扩展结点，就**一次性产生其所有儿子结点**。在这些儿子结点中，导致不可行解或导致非最优解的儿子结点被**舍弃**，其余儿子结点被加入**活结点表**中。
- 从活结点表中取下一结点成为当前扩展结点。这个过程一直持续到找到所需的解或活结点表为空时为止。
- 取下一结点时，按最小代价优先的方式，成为当前扩展结点；并按**估值函数和约束条件**进行剪枝。

分枝-限界方法找最优解的效率比回溯法高。其原因在于它采用了最小代价估值函数指导搜索。在活结点表中，选择有最小代价估值的结点作为扩展结点。即总是向最有可能获得最优解的子树上扩展，并且采用限界函数U杀死活结点表中不可能成为最优解的结点，提高算法的效率。

分枝-限界方法运用于实际问题最关键的是选取有效的最小代价估值函数。并且要把极大化问题转化成极小化问题来解决。

作业：

- 1.什么是成本估值函数？
- 2.LC搜索与FIFO、LIFO有什么不同？
- 3.LC检索的特性是什么？
- 4.对9宫格和15谜问题找最小成本路线。
- 5.分枝-限界算法中是如何对搜索加速？
- 6.分枝-限界主要解决的是什么？
- 7.分枝-限界中扩展结点的策略是什么？
8. $c(x)$ 与 $\hat{c}(x)$ 、 U 的关系是怎样的？在何时杀死结点？说明了什么？